

PRISM — Đặc tả pipeline (bản chi tiết)

Provenance-anchored · Reliability-calibrated · Image-verified Sentiment Monitoring — aspect-level sentiment drift trên 1,95M review Booking.com.
commit **fc5c774** (2026-09-07) · biên soạn 2026-09-08 · scope: **prism/src/prism**, **prism/scripts**, **prism/kaggle_notebooks**, **prism/tests**, **prism/docs**, Docker · quy ước trích dẫn vị trí **theo tên module/hàm**, không dùng số dòng (số dòng đổi theo commit).

Tài liệu này là **đặc tả thực thi**: mọi thứ trong đây được đọc trực tiếp từ code ở commit **fc5c774** — chữ ký hàm, giá trị mặc định của từng cờ CLI, schema từng file output, công thức đúng như đang cài, và những chỗ **code khác tài liệu phương pháp** (được đánh dấu **LỆCH**). Ba nhóm người đọc: (1) người chạy pipeline cần biết chạy gì, theo thứ tự nào, cần file gì; (2) người viết bài cần biết con số nào là con số chính thức và nó được tính bằng công thức nào; (3) người refactor cần biết bất biến nào không được phá. Bằng chứng thực nghiệm và giao thức đánh giá nằm ở §14–§16, phần vận hành (Kaggle · Docker · test · bất biến) ở §17–§19, và phụ lục A–D là tra cứu nhanh (chỉ mục CLI · bản đồ file · số liệu đã đo · thuật ngữ).

Bản sửa đổi — đồng bộ với vòng sửa code (xem docs/prism_code_review.pdf)

Tài liệu này đã được cập nhật sau khi áp dụng 28 hạng mục sửa từ review code. Mọi chỗ đổi hành vi được đánh dấu **đã đổi**. Tóm tắt những thay đổi **ảnh hưởng tới cách đọc số**:

§	Đổi gì	Ảnh hưởng tới số
§5.2	Khoá blacklist (hotel,date) phát hiện là rỗng trên checkout hiện tại	● flag_gold_leak 18.475 → 1.466; ~17.000 dòng lẻ ra bị chặn đang lọt vào training
§13.5	p_perm nay là kiểm định TREND ; bước nhảy tách sang p_perm_changepoint	● n_significant_after_fdr trước đây đếm changepoint nhưng được bảo cáo cạnh slope OLS
§13.2	D-a vẫn là stub, nhưng nay log WARNING , ghi recall_adjusted: false , và --final thì dừng	Bản cũ log “áp dụng recall_table” trong khi không áp gì
§12.5	Go/no-go được cưỡng chế trong code : NO-GO ghi bridge_NOGO.pkl và xoá bridge.pkl	Bản cũ vẫn dùng bridge NO-GO cho w của toàn corpus
§8.3	Bucket by_language["other"] nay là phần bù của en/vi	Bản cũ bucket này luôn rỗng và mọi review không en/vi bị bỏ đếm
§6.2	Term chứa / <quad> được escape	Bản cũ mất trắng quad đó — recall lệch theo <i>loại term</i> , không hiện ở bảng nào
§8.4	Probe E1c phát hiện chưa từng chạy được (0/10.711 segment có ngày)	● chrono_train từng chứa toàn bộ train+dev, chrono_test rỗng
§10.3	Rescore bị truncate nay được đánh cờ p_model_truncated	Quad trong unit nhiều quad từng có posterior = prior thuần, im lặng
§14.3	inject_shuffle xáo ở mức review	● E4 trước đây ước lượng FPR thấp vì phá tương quan trong review
§17.4	Bỏ hẳn hai bản và source lúc chạy	—
§19	61 test (thêm tests/test_regressions.py)	—

Chưa đóng được: §5.2 và §8.4 chờ **metadata/reviews_with_dates.jsonl**; §13.2 chờ tập audit D0.

Mục lục

Phần I — Khung chung

- §1 · Bài toán, chuỗi giá trị, kiến trúc 4 module
- §2 · Cấu hình tập trung (**config.py**)
- §3 · Dữ liệu đầu vào & audit số đo
- §4 · Tiện ích dùng chung (**utils.py**)

Phần II — Bốn module

- §5 · Module A — Temporal Quad Store
- §6 · Module B-data — text-to-text
- §7 · Module B-train — seed extractor mT5
- §8 · Module B-eval — E1/E1b/E1c
- §9 · Module B-selftrain — teacher→student
- §10 · Module B-infer — pool + provenance posterior
- §11 · Ảnh pool & hai mẫu annotation người
- §12 · Module C — Cross-modal reliability [go/no-go]

§13 · Module D — Dual-bias compositional drift

Phần III — Đánh giá & vận hành

- §14 · Injection & negative control (E3/E4)
- §15 · Bảng chứng đã có (probe CACT)
- §16 · Giao thức đánh giá, ablation, baseline, rủi ro
- §17 · Orchestrator Kaggle & chuỗi notebook
- §18 · Môi trường: local · Docker · Kaggle
- §19 · Test, smoke test, bất biến, trạng thái & việc còn thiếu
- §20 · **Chạy trên Vast.ai — cấu hình cho kết quả khoa học nhất**

Phụ lục

- A · Chỉ mục CLI đầy đủ
- B · Bản đồ file toàn repo
- C · Tra cứu số liệu đã đo
- D · Ký hiệu & thuật ngữ

§1 · Bài toán, chuỗi giá trị, kiến trúc

1.1 Bài toán và đại lượng cần ước lượng

Câu hỏi nghiên cứu: **theo thời gian, khách phàn nàn về aspect nào nhiều hơn, và khi nhắc tới một aspect thì họ chê nặng hơn không?** — ước lượng trên ~1,15M review có text (không thể annotate hết), bằng một extractor có recall lệch theo cực tính/độ dài, trên một tổng thể mà **thành phần người viết review thay đổi rất mạnh** (reviewer Việt Nam 89,7% → 16,1% trong 36 tháng).

Hai đại lượng đích, tính theo từng kỳ tháng **t** và từng aspect **a** :

$\pi_{a,t}$ = share-of-complaints — tỷ trọng của aspect a trong tổng lượt phản nản của kỳ t , phân tích trên thang CLR.

Trả lời: “khách đang phản nản về cái gì nhiều hơn?”. Là dữ liệu compositional (cộng lại = 1) nên buộc phải qua CLR.

$v_{a,t}$ = negativity rate = $P(\text{negative} \mid \text{có nhắc } a, \text{ kỳ } t)$ — trên **mọi** quad, direct standardization theo strata.

Trả lời: “nhắc tới a thì có chê nhiều hơn không?”. **Đây là estimand trung tâm của bài.**

Mỗi đại lượng có **hai bản**: **raw** (thô) và **adj** (đã hiệu chỉnh thành phần). Con số chính thức của paper là bản **adj sau BH-FDR**; bản raw luôn được báo cáo kèm để minh bạch (§13.7, §19.4).

1.2 Chuỗi giá trị

gold 23.995 quad → train extractor mT5 → chạy trên pool 1,9M review chưa nhãn
→ gán độ tin cậy từng quad (cross-modal, dùng ảnh) → drift 2 kênh có hiệu chỉnh thành phần + kiểm định (permutation null riêng từng chuỗi, BH-FDR, bootstrap CI)

1.3 Bốn module & thứ tự bất biến

Thứ tự sau **không đảo được** vì phụ thuộc dữ liệu:

A store → B-data → B-train → (B-eval) → B-selftrain → B-infer → (photos)
→ C(train_verifier → apply_verifier → bridge → apply) → D drift → [injection]

Module	Tên	Vai trò một câu	File	Máy	Trạng thái
A	Temporal Quad Store	Biến pool thô 1,95M dòng thành corpus sạch có trục thời gian, chống leakage gold, gán stratum/cohort	module_a_store.py	CPU	đã chạy thật
B	Provenance-Anchored Extraction	mT5 sinh quad (aspect, code, opinion, sentiment) + posterior kết hợp prior bất đối xứng theo ô nguồn trường	module_b_{data,train,eval,selftrain,infer}.py	GPU	seed đã train
C	Cross-Modal Reliability Calibration	Dùng ảnh làm “annotator miễn phí” trên tập con có ảnh → hiệu chỉnh estimator chỉ-text → áp cho toàn bộ quad	module_c_reliability.py	GPU / CPU	chưa chạy · có go/no-go
D	Dual-Bias Compositional Drift	Tách lỗi đo của công cụ (D-a recall) khỏi đối thành phần tổng thể (D-b composition); 2 kênh × 2 bản + kiểm định	module_d_drift.py	CPU	khung đã kiểm chứng

1.4 Sơ đồ phụ thuộc dữ liệu

```
data/raw/*.jsonl + hamos-mabsa/ (gold quad · segment · ảnh)
├── (A) module_a_store ──────────> outputs/store/reviews.jsonl.gz
│                               outputs/store/hotel_cohorts.json
│                               outputs/store/store_report.json
│                               | store là input của selftrain, infer, photos
├── (B-data) module_b_data ───> outputs/extract/{train,dev,test}.t2t.jsonl
│                               outputs/extract/chrono_{train,test}.t2t.jsonl
│
│   (B-train) ──────────────────> models/seed_extractor/ (+ training_log.json)
│
│   (B-eval)  ← ckpt + t2t ───> outputs/extract/eval_report.json [tùy chọn]
│
│   (B-selftrain) ← seed + store ─> models/selftrain_round<r>/
│                               outputs/extract/selftrain_history.json (final_ckpt)
│
│   (B-infer)  ← final_ckpt + store ─> outputs/extract/pool_quads.<cohort>.jsonl.gz
│
│   (photos)  ← store ──────────> outputs/reliability/pool_images/*.jpg
│                               outputs/reliability/pool_image_index.json
│
│   (C1) train_verifier ← gold ảnh ─> reliability/verifier.pkl + verifier_report.json
│   (C2) apply_verifier ← pool_quads + image_index ─> reliability/pool_quads_vimg.<cohort>.jsonl.gz
│   (C3) bridge        ← vimg + audit_sample_300 ─> reliability/bridge.pkl + bridge_report.json
│   (C4) apply         ← pool_quads + bridge.pkl ─> reliability/quads_weighted.jsonl.gz
│
│   (D) module_d_drift ──────────> outputs/drift/drift_results.<cohort>.<level>.json
│   (E) eval_injection ──────────> outputs/drift/injection_{composition,valence,shuffle}.json
```

1.5 Ma trận step: máy, thời lượng, phụ thuộc

#	Step	Entrypoint	Máy	Thời lượng thực tế	Chặn bởi
1	unit test	python -m unittest discover tests	CPU	~0,15 s · 61 test	—

2	smoke test	<code>python -m prism.smoke_test</code>	CPU	~2 phút (dữ liệu thật)	data/raw + hamos
3	store	<code>python -m prism.module_a_store</code>	CPU	~10 phút · ra 180 MB	pool + splits + gold meta
4	data	<code>python -m prism.module_b_data</code>	CPU	~10 s	splits + gold meta
5	audit D0	<code>python -m prism.make_audit_samples</code>	CPU	~2 phút	store
6	train	<code>python -m prism.module_b_train</code>	GPU	~66 phút (mt5-small, T4, 10 epoch)	t2t
7	eval	<code>python -m prism.module_b_eval</code>	GPU tùy chọn	~10 phút / 1.890 segment	ckpt + t2t
8	selftrain	<code>python -m prism.module_b_selftrain</code>	GPU	rất nặng — xem §9.5	seed ckpt + store + t2t + dev.jsonl
9	infer	<code>python -m prism.module_b_infer</code>	GPU	step nặng nhất — xem §10.7	final_ckpt + store
10	photos	<code>python scripts/download_pool_photos.py</code>	NET	~0,4 s/ảnh (rate-limit)	store
11	audit AUDIT	<code>make_audit_samples --quads ...</code>	CPU	~1 phút	pool_quads
12	c_verifier	<code>module_c_reliability --stage train_verifier</code>	GPU	~9.219 ảnh embed một lần	hamos-mabsa (ảnh gold)
13	c_apply_verifier	<code>--stage apply_verifier</code>	GPU	tỷ lệ theo số ảnh pool đã tải	verifier.pkl + image_index + pool_quads
14	c_bridge	<code>--stage bridge</code>	CPU	~giây	file <code>vimg</code> (bắt buộc có <code>v_image</code>)
15	c_apply	<code>--stage apply</code>	CPU	tuyến tính theo số quad	bridge.pkl + pool_quads
16	drift	<code>python -m prism.module_d_drift</code>	CPU	phút → giờ (bootstrap 1000)	quads_weighted hoặc pool_quads
17	injection	<code>python -m prism.eval_injection</code>	CPU tùy chọn	= n lần chạy drift	quads_weighted

§2 · Cấu hình tập trung — `src/prism/config.py`

Nguyên tắc: **mọi hằng số và ngưỡng nằm ở một file**, không rải rác trong code. Giá trị có chú thích `[đo]` trong source là số đã audit trực tiếp trên dữ liệu, không phải đặt tay.

2.1 Gốc đường dẫn & biến môi trường override

Hằng	Mặc định	Env override	Ghi chú
TABSA_ROOT	parents[2] của config.py (= gốc repo prism/)	PRISM_TABSA_ROOT	src-layout: <code><root>/src/prism/config.py</code>
HAMOS_ROOT	TABSA_ROOT.parent / "hamos-mabsa"	PRISM_HAMOS_ROOT	repo gold quad/ảnh, đặt cạnh repo này
WORK_DIR	TABSA_ROOT/outputs	PRISM_WORK_DIR	gốc mọi output pipeline
MODEL_DIR	TABSA_ROOT/models	PRISM_MODEL_DIR	checkpoint HF
STORE_DIR	outputs/store	—	Module A
EXTRACT_DIR	outputs/extract	—	Module B
RELIAB_DIR	outputs/reliability	—	Module C + mẫu audit + ảnh pool
DRIFT_DIR	outputs/drift	—	Module D + injection + <code>recall_table.json</code>

Cơ chế tìm file gold — `_hamos_file(name, fallback)` thử lần lượt `HAMOS_ROOT/data/<name>` rồi `HAMOS_ROOT/<name>`, nên chấp nhận cả layout `hamos-mabsa/data/...` (repo gốc) và layout phẳng `hamos-mabsa/...` (khi attach trên Kaggle). Riêng `GOLD_META` có **chuỗi fallback 3 tầng**: `metadata/reviews_with_dates.jsonl` → `metadata/reviews.jsonl` → `data/raw/hotel_absa_labeled.jsonl` (bản local, chỉ metadata, **không** chứa quad).

POOL_JSONL	data/raw/hotel_booking_unlabeled.jsonl — pool 1.949.604 review
SPLIT_DIR	data/raw/ — chứa {train,dev,test}.jsonl
GOLD_QUADS	annotations/quads.jsonl (23.995 quad)
GOLD_SEGS	metadata/segments.jsonl
GOLD_IMAGES	metadata/images.jsonl — có <code>local_path</code> từng ảnh
IMAGE_DIR	images/ — 9.219 ảnh gold (986 MB)

2.2 Cửa sổ thời gian

Hằng	Giá trị	Ví sao
WINDOW_START	"2022-03"	2022-02 chỉ có 1.391 review / 549 hotel — tháng dở dang
WINDOW_END	"2025-02"	2025-03 chỉ có 11.211 review (crawl dừng ngày 10/03)

→ **36 kỳ tháng** (kiểm chứng bằng test `test_periods_in_window_36_months`). Cắt hai biên là bắt buộc: tháng dở dang tự sinh "drift" giả ở đầu/cuối chuỗi.

2.3 Taxonomy

6 category · 31 taxonomy code (`CODE2CAT` , `ALL_CODES = sorted(CODE2CAT)`) · 3 sentiment (`positive` · `neutral` · `negative`).

Category	Code thuộc category	Số code	% quad gold
FACILITY	FAC_ROOM · FAC_VIEW_LOCATION · FAC_BUILDING · FAC_BATH · FAC_ENV · FAC_INTERIOR · FAC_CLIMATE · FAC_SECURITY	8	55,1%
AMENITY	AM_POOL · AM_FOOD · AM_ROOM_UTIL · AM_TRANSPORT · AM_WELLNESS · AM_ENT · AM_WIFI · AM_UTILITY	8	20,5%
SERVICE	SER_ATTITUDE · SER_SUPPORT · SER_OPERATION · SER_COMM · SER_PROFESSIONALISM	5	10,7%
EXPERIENCE	EXP_OVERALL · EXP_VALUE · EXP_EMOTION · EXP_SAFETY	4	8,3%
LOYALTY	LOY_RETURN · LOY_RECOMMEND · LOY_PREFERENCE	3	3,2%
BRANDING	BRA_CONSISTENCY · BRA_LUXURY · BRA_REPUTE	3	2,2%

CODES_REPORTABLE — **14 code có ≥300 quad gold**, được báo cáo drift riêng từng code: `FAC_ROOM` · `FAC_VIEW_LOCATION` · `AM_POOL` · `FAC_BUILDING` · `SER_ATTITUDE` · `AM_FOOD` · `FAC_BATH` · `EXP_OVERALL` · `AM_ROOM_UTIL` · `FAC_ENV` · `LOY_RETURN` · `FAC_INTERIOR` · `EXP_VALUE` · `SER_SUPPORT`. 17 code còn lại (kể cả `BRA_CONSISTENCY` 286 và `LOY_RECOMMEND` 262 — sát ngưỡng) **tự động gộp lên category cha** tại Module D (§13.2). Lý do: dev có `BRA_REPUTE` n=1, test có `AM_UTILITY` n=5 → F1 per-code vô nghĩa với các code hiếm.

2.4 Provenance prior (Module B)

Đo trên **14.129 quad gold** xác định được ô nguồn trường (`review_positive` / `review_negative`):

Segment lấy từ ô	n	→ positive	→ neutral	→ negative
<code>review_positive</code> (φ=POS)	11.733	0,931	0,036	0,033
<code>review_negative</code> (φ=NEG)	2.396	0,217	0,142	0,641

Bất đối xứng 0,931 vs 0,641 là phát hiện, không phải nhiễu: khách thường viết nội dung tích cực ngay trong ô “chưa hài lòng” (“*The owner was lovely although...*”) và annotation gán positive ở đó là **đúng**. Vì vậy prior **không được giả định đối xứng** và **không được dùng làm bộ lọc cứng** — lọc cứng sẽ vớt đúng 21,7% ca khó và giàu thông tin nhất. **PROVENANCE_LAMBDA = 0.7** (trọng số model vs prior; **phải quét lại trên dev** $\lambda \in \{0,5...0,9\}$ trước run chính thức — xem §19.5).

2.5 Strata & ngưỡng ô (Module D)

Hằng	Giá trị	Ý nghĩa
COUNTRY_BLOCS	VN · WEST · ASIA · OTH	4 khối quốc tịch reviewer; WEST_COUNTRIES 23 nước, ASIA_COUNTRIES 13 nước (so khớp chuỗi tiếng Việt đã hạ chữ, có cả 2 biến thể chính tả “thuy/thuy.”)
TRAVELLER_TYPES	Cặp đôi · Phòng gia đình · Khách lẻ · Nhóm · NA	[đo] 4 loại chiếm 99,7% (40,8 / 24,0 / 18,9 / 16,0%)
LENGTH_BINS	[(0,15), (15,45), (45,1e9)]	L0 / L1 / L2 theo số từ
USE_LENGTH_IN_STRATA	False	False → 20 ô (4×5); True → 60 ô (ablation robustness)
MIN_STRATUM_N	30	Kênh π : cường độ prior shrinkage của ô nhỏ (không phải ngưỡng loại — §13.3)
MIN_VALENCE_CELL_W	10	Kênh v : cường độ prior Beta khi shrink ô (kỳ × stratum × aspect)

Lịch tài liệu ↔ code **LỊCH**

docs/approach_temporal_trend.md và bản PDF spec cũ ghi “ô $n < 30$ **bị bỏ qua**”. Code hiện tại (sau changelog 2026-08-22) **không loại ô nhỏ nữa** mà **shrink về share/tỷ lệ gộp của kỳ** với cường độ `min_n` / `min_w`. Lý do đổi: loại ô đột ngột làm mix strata đóng góp tự trôi theo thời gian khi một stratum teo dần → **sinh trend giả ngay trong kênh adj** (đã tái hiện được bằng composition injection trên nền null).

2.6 Tham số thống kê

Hằng	Giá trị	Dùng ở đâu
N_BOOTSTRAP	1000	CI slope kênh π -adj (resample review trong strata) — lưu ý CLI mặc định --n-boot 0 , phải truyền tay 1000 cho kết quả cuối
N_PERMUTATION	1000	permutation null riêng từng chuỗi
FDR_ALPHA	0,05	BH-FDR + tiêu chí PASS của E4
T_THRESHOLD	2,5	Ngưỡng t “có ý nghĩa” trước FDR — chỉ dùng để đọc bảng probe (§15), không nằm trong máy suy diễn của Module D
RANDOM_SEED	20260821	seed chung mọi module (train, sampling, permutation, bootstrap, injection)

2.7 Cohort **đã đổi**

COHORT_DEFS nay là **nguồn sự thật duy nhất**: `module_a_store.build_store` đọc dict này thay vì hardcoded `>= 1000 / >= 300 / sp == "test"`, và `store_report.json` ghi kèm `cohort_defs` đã dùng. Trước đây dict này là **code chết** — sửa `min_pool` trong config không đổi gì, đúng ở cái knob để bị tinh chỉnh nhất. Mỗi entry nay đủ 3 khoá (`needs_gold`, `min_pool`, `gold_split`) — `T-unbiased` trước đây thiếu `needs_gold`. Đã verify: chạy lại Module A cho cohort **khớp tuyệt đối** 270 / 1.207 / 514 / 10.629.

Cohort	Định nghĩa (COHORT_DEFS)	n hotel [config]	n hotel [run thật]	Dùng cho
A-dense	có gold $\wedge \geq 1000$ review pool	276	270	drift cấp hotel, granularity tháng
B-anchor	có gold $\wedge \geq 300$ review pool	1.221	1.207	drift cấp hotel granularity quý; cohort mặc định của self-train
T-unbiased	<code>gold_split == "test"</code>	514	514	cohort đánh giá không thiên vị — extractor chưa từng thấy hotel này (E5)
corpus	toàn bộ (danh sách rỗng = không lọc)	10.631	10.629	kết quả chính — nơi tín hiệu mạnh nhất

Chênh lệch nhỏ (276→270, 1.221→1.207, 10.631→10.629) là do `pool_count` trong Module A chỉ đếm các dòng **đã qua** dedup + cắt cửa sổ, còn số trong `config.py` đếm trên pool thô. Con số dùng để báo cáo là con số trong `outputs/store/store_report.json`.

2.8 RunConfig & ensure_dirs

RunConfig là dataclass ghi kèm output để tái lập: `window_start`, `window_end`, `use_length_in_strata`, `min_stratum_n`, `n_bootstrap`, `n_permutation`, `fdr_alpha`, `seed`, `provenance_lambda`, `encoder_version`, `extractor_ckpt`, `notes` + `to_dict()`. `ensure_dirs()` tạo sẵn 6 thư mục output (WORK/STORE/EXTRACT/RELIAB/DRIFT/MODEL) và được gọi ở đầu mọi module có ghi file.

Bất biến — mọi run phải ghi config / args vào output JSON

Module D ghi `vars(args)` vào khoá `"config"`; B-train ghi `vars(args)` + `history` vào `training_log.json`; C ghi `thresholds` + `encoder_version` vào `verifier_report.json`. Không ghi → không tái lập được → kết quả không dùng được cho paper.

§3 · Dữ liệu đầu vào & audit số đo

3.1 Bốn nguồn tiền quyết

Nguồn	Đường dẫn	Quy mô	Vai trò
Pool chưa nhãn	<code>data/raw/hotel_booking_unlabeled.jsonl</code>	1.949.604 dòng · 1,4 GB · 10.631 hotel · 0 dòng JSON lỗi	Input duy nhất của Module A
Gold split	<code>data/raw/{train,dev,test}.jsonl</code>	8.816 / 1.895 / 1.890 segment · 23.995 quad · hotel-disjoint (2.381/504/514 hotel, overlap 0)	Train/val/test extractor + dựng blocklist + dựng cohort
Gold meta	<code>hamos-mabsa/.../reviews_with_dates.jsonl</code> (fallback <code>hotel_absa_labeled.jsonl</code>)	8.796 dòng	<code>review_date</code> , <code>languages</code> , <code>review_text</code> (cho blocklist hash) — không chứa quad
Gold quad/segment/ảnh	<code>hamos-mabsa/</code> qua <code>PRISM_HAMOS_ROOT</code>	23.995 quad · 12.601 segment · 9.219 ảnh (986 MB)	Module B-data (nhãn) + Module C1 (ảnh gold)

Mỗi dòng gold split: `instance_id · source_review_id · segment_id · text · review_date · quads[]`; mỗi quad = `(aspect_term, taxonomy_code, aspect_category, opinion_term, sentiment)` + span.

3.2 Trường của pool: độ phủ & giá trị nghiên cứu

Trường	Kiểu thô	Độ phủ	Vai trò
<code>hotel_id</code>	str	100%	Đơn vị cấp hotel. Gold dùng int → phải ép kiểu khi join (Module A ép về str)
<code>review_date</code>	str VI: "Ngày đánh giá: ngày D tháng M năm YYYY"	99,99% (184 null)	Trục thời gian chính — không có định dạng ISO trong pool, phải parse regex
<code>review_score</code>	str "Đạt điểm 9,0" (dấu phẩy thập phân)	100%	Weak signal + biến detrend. Lệnh dương nặng: điểm 10 = 44,9%; TB tháng chỉ dao động 8,47–8,87
<code>review_positive</code>	str null	57,6%	Text nguồn + nhãn polarity ngầm φ =POS
<code>review_negative</code>	str null	36,2%	Text nguồn + φ =NEG. Nguồn quad negative chính
<code>country</code>	str	99,7%	Confounder quan trọng nhất : VN 29,9% · Pháp 6,8% · Úc 6,6% · Đức 6,5% · Anh 6,4%
<code>state</code>	str	99,7%	Loại khách — confounder chủ lực thứ hai
<code>room</code>	str	99,7%	Cho phép kiểm soát "đổi mix sản phẩm" thay vì "đổi chất lượng"
<code>stars_rating</code>	float null	89,9%	Hạng sao hotel — biến phân tầng
<code>review_photo</code>	dict {url: caption}	6,24% (121.584)	Nhánh multimodal — nguồn của Module C2
<code>date</code>	str "1 đêm · Tháng 2/2025"	99,7%	Ngày lưu trú (khác ngày review) — cho phép tính độ trễ review
<code>name</code>	str	99,7%	Tên reviewer. Không có reviewer_id → không loại được reviewer lặp
<code>time_crawl</code>	str	100%	Toàn bộ ~2025-03 → xác nhận crawl một lần

Không tồn tại trong dữ liệu (phải khai báo với reviewer): `reviewer_id`, địa lý hotel dạng cấu trúc, giá phòng, thông tin renovation/đổi chủ, review response của hotel, helpfulness vote, dữ liệu nền tảng khác ngoài Booking.com.

3.3 Bốn confounder trôi theo thời gian (lý do tồn tại của Module D)

Thứ trôi	Biên độ đo được	Phá hỏng đại lượng nào	π tự khử?
Số hotel/tháng	549 (2022-02) → 8.535 (2025-02) — 15,6×	count tuyệt đối	✓ (log-ratio bất biến với tổng khối lượng)
% review có text	42,1% → 63,4%	count tuyệt đối	✓
Độ dài review TB	27,5 → 40,8 từ	count và recall của extractor	✓ phần recall chung; Δ phần recall lệch theo độ dài cần D-a
Reviewer Việt Nam	89,7% → 16,1% (2022-03 → 2025-01)	mọi tỷ lệ chưa hiệu chỉnh	✗ bắt buộc hiệu chỉnh tường minh (D-b)

Vì vậy: **không dùng count, không dùng điểm trung bình** (score gần phẳng 8,47–8,87 và lệnh dương nặng — không đủ độ phân giải), mà dùng **share + standardization**.

3.4 Gold: cấu trúc & giới hạn

Chiều	Số đo	Hệ quả cho pipeline
Quy mô	23.995 quad · 12.601 segment · 8.796 review · 3.399 hotel · 38 tháng	Cơ sở train extractor

Sentiment	positive 79,4% · negative 15,4% · neutral 5,2% (1.248)	Mất cân bằng nặng; neutral là lớp yếu nhất. 15,4% chính là mốc gold_neg_rate của guardrail self-train
Ngôn ngữ	en 7.473 · vi 1.032 · other 205 · đa ngữ 86 → 80,5% en · 16,2% vi	Nghịch đảo so với pool (~30% có dấu tiếng Việt) → chọn mT5, cân nhắc oversample vi
Quad/review · quad/segment	mean 2,73 (max 36) · mean 1,90 (max 23)	Cơ sở đặt --max-tgt 192
Độ dài segment	mean 13 từ · max 147	Cơ sở đặt --max-src 160
Implicit	aspect 3,3% (782) · opinion 1,2% (283)	Không làm claim về implicit (chuẩn ACOS ~30%)
Phủ văn bản	Gold chỉ chú thích 26% văn bản gốc; phủ ô positive 42% nhưng negative chỉ 25%	Nguồn của D-a: recall lệch theo cực tính
Đa modal	100% quad có primary_image_id ; 9.175 ảnh là primary	Nhưng grounding chỉ ở mức review — xem §12.2

3.5 Overlap gold ↔ pool: vì sao blacklist là bắt buộc

- **3.399/3.399 (100%)** hotel gold có mặt trong pool.
- **1.249.424** review pool (64,1%) thuộc hotel có gold → đây là universe làm việc của cohort A/B.
- Đã xác nhận **gold ⊂ pool** bằng ví dụ cụ thể: **H3977209_R00003** (2022-09-05).

R2 — leakage gold vào pool **P1**

Nếu không chặn: self-training sẽ **nuốt cả test set** → mọi F1 báo cáo trở nên vô hiệu và không phát hiện được bằng cách đọc kết quả. Đây là lý do Module A có bước **A4 blacklist [BLOCKING]** với 3 khoá (§5.2), và lý do **module_b_infer** bỏ qua vô điều kiện mọi dòng **in_gold=true** .

3.6 Granularity thời gian cho phép

Cấp phân tích	Granularity	Bảng chứng số
Corpus	Tháng	~60,5k quad/tháng ước tính — dư sức cho cả subcategory hiếm
Cohort (≥50 hotel)	Tháng	Luôn đủ dày
Hotel ≥1000 review (323 hotel)	Tháng	~26 review/tháng → FAC_ROOM ~7,5 quad/tháng
Hotel 300–1000 review (1.370)	Quý	~8 review/tháng → FAC_ROOM ~2,3 quad/tháng, quá thừa cho tháng
Hotel <300 review (8.938)	Không phân tích riêng	median 66 review / 38 tháng = 1,7 review/tháng

Không dùng granularity tuần ở bất kỳ cấp nào: chu kỳ ngày-trong-tuần và kỳ nghỉ lẫn át tín hiệu.

§4 · Tiện ích dùng chung — `src/prism/utils.py`

Mọi module import `from . import utils as U`. Không có phụ thuộc ngoài `stdlib`.

4.1 I/O & logging

Hàm	Hợp đồng
<code>get_logger(name)</code>	Logger stderr một handler, format <code>[HH:MM:SS] name LEVEL msg</code> , level INFO. Idempotent (không nhận handler khi gọi lại)
<code>read_jsonl(path)</code>	Iterator (không nạp cả file vào RAM). Tự mở <code>gzip</code> nếu đuôi <code>.gz</code> ; bỏ dòng trống
<code>write_jsonl(path, rows, gzip_out=False)</code>	Ghi từng dòng, <code>ensure_ascii=False</code> , tự tạo thư mục cha, tự gzip nếu đuôi <code>.gz</code> . Trả về số dòng đã ghi
<code>write_json(path, obj)</code>	JSON indent 2, UTF-8 giữ nguyên dấu

4.2 Chuẩn hoá văn bản & định danh

Hàm	Hợp đồng	Dùng ở đâu
<code>nfc(s)</code>	NFC normalize + strip; <code>None</code> → ""	Chuẩn hoá text pool/gold khi đọc
<code>norm_text(s)</code>	Chuẩn hoá mạnh để so khớp: NFKC + hạ chữ + bỏ dấu câu + gộp khoảng trắng	Blocklist hash, khoá dedup, exact-match của B-eval
<code>text_hash(s)</code>	MD5 của <code>norm_text(s)</code>	Dedup A3, blocklist khoá text
<code>quad_uid(q)</code>	<code>f"{review_uid}_{phi}_{taxonomy_code}_{text_hash(opinion_term)[:6]}"</code>	Khoá join giữa mẫu audit người và Module C bridge

Bất biến — `quad_uid` phải dùng chung một hàm

`make_audit_samples` sinh khoá và `module_c_reliability.fit_bridge` dựng lại khoá để đối chiếu. Hai bên tự dựng khoá riêng thì **không khớp cặp nào** → Spearman audit không tính được → go/no-go của Module C tưởng là NO-GO. Bug này đã từng xảy ra và được sửa (changelog 2026-08-22); đừng inline lại công thức khoá ở nơi khác.

4.3 Parser dữ liệu thô (Việt hoá)

Hàm	Regex / logic	Trả về
<code>parse_review_date(raw)</code>	Ưu tiên <code>ngày\s+(\d{1,2})\s+tháng\s+(\d{1,2})\s+năm\s+(\d{4})</code> ; fallback ISO <code>(\d{4})-(\d{2})-(\d{2})</code> ; fallback <code>(\d{1,2})/(\d{1,2})/(\d{4})</code> . Kiểm tra $1 \leq \text{tháng} \leq 12, 1 \leq \text{ngày} \leq 31$	<code>(iso_date, period="YYYY-MM")</code> hoặc <code>None</code> . [đo] 99,99% pool khớp mẫu VI; 184 dòng null
<code>parse_score(raw)</code>	Bỏ tiền tố "Đạt điểm", khớp <code>(\d{1,2})([.,](\d{0,2}))?</code> , ghép thành float; chấp nhận int/float truyền thẳng	float ∈ [0,10] hoặc <code>None</code> (loại giá trị ngoài khoảng)
<code>parse_nights(raw)</code>	<code>(\d+)\s*đêm</code>	int hoặc <code>None</code>

4.4 Strata & trực thời gian

Hàm	Hợp đồng
<code>country_bloc(country)</code>	"việt nam" → VN; ∈WEST_COUNTRIES→WEST; ∈ASIA_COUNTRIES→ASIA; còn lại (kể cả <code>None</code>)→OTH
<code>traveller_type(state)</code>	Giữ nguyên nếu ∈ <code>TRAVELLER_TYPES</code> , ngược lại "NA"
<code>length_bin(n_words)</code>	L0 / L1 / L2 theo <code>LENGTH_BINS</code>
<code>make_stratum(country, state, n_words, use_length=USE_LENGTH_IN_STRATA)</code>	Tuple <code>(bloc, traveller)</code> , cộng thêm <code>length_bin</code> nếu <code>use_length</code>
<code>in_window(period) · periods_in_window()</code>	So sánh chuỗi "YYYY-MM" (đúng thứ tự từ điển = đúng thứ tự thời gian); sinh danh sách 36 kỳ

4.5 Máy thống kê

`clr(shares, eps=1e-6)` — centered log-ratio: $\text{clr}(p)_a = \log(p_a / (\prod_a' p_{a'})^{1/K})$, sau khi kẹp $p_a \geq \text{eps}$.

Bắt buộc: các share cộng lại = 1, nên một aspect tăng thì các aspect khác **bắt buộc phải** giảm → hồi quy trực tiếp trên share sinh tương quan âm giả và vi phạm ràng buộc simplex. CLR đưa từ simplex về không gian thực (tổng các thành phần $\text{clr} = 0$ — có unit test kiểm chứng) và hiệu log-ratio **bất biến với tổng khối lượng** — đúng thứ cần khi hotel/tháng tăng 15,6×.

`benjamini_hochberg(pvals, alpha)` — trả q-value **giữ đúng thứ tự input**: quét từ p lớn nhất xuống, $q_{(i)} = \min(q_{(i+1)}, p_{(i)} \cdot n/i)$, kẹp ≤ 1 .

Cài đặt "step-up có monotone enforcement". Test kiểm chứng ví dụ chuẩn [0,01...0,04; 0,9] → 4 giá trị đầu đều đúng 0,05.

welch_t(a, b) = (mean a – mean b) / $\sqrt{(\text{var } a/n_a + \text{var } b/n_b)}$, với var là ước lượng không chệch (chia n–1).

Trả **0,0** khi $n < 2$ ở một phía hoặc mẫu số = 0 (hai nửa phương sai 0) — guard này khiến changepoint không bao giờ chia cho 0, nhưng cũng có nghĩa chuỗi hằng số hoàn toàn cho $t=0$.

\$5 · Module A — Temporal Quad Store CPU

src/prism/module_a_store.py · chạy: `python -m prism.module_a_store` · không cần torch · ~10 phút · đã chạy thật 2026-08-21

Nhiệm vụ: biến 1.949.604 dòng pool thô thành corpus có trục thời gian sạch, **chống leakage gold**, và gắn sẵn mọi trường mà Module B/C/D cần (period · stratum · cohort · has_photo · in_gold). Đây là step **chặn mọi thứ**: sai ở đây thì mọi kết quả phía sau vô hiệu.

5.1 Input / Output

Chiều	File	Ghi chú
IN	data/raw/hotel_booking_unlabeled.jsonl	pool thô
	data/raw/{train,dev,test}.jsonl	chỉ lấy <code>source_review_id</code> → map review → split
	GOLD_META	lấy <code>hotel_id</code> , <code>review_date</code> , <code>review_text</code> để dựng blocklist
OUT	outputs/store/reviews.jsonl.gz	180 MB · 1.921.661 dòng · schema \$5.4
	outputs/store/hotel_cohorts.json	<code>{cohort: [hotel_id...]}</code> ; corpus là danh sách rỗng = không lọc
	outputs/store/store_report.json	stats + cohort_sizes + window + gold_leak_flagged

5.2 A4 — Gold blocklist [BLOCKING]: ba khoá

`build_gold_blocklist()` trả về 3 cấu trúc:

#	Khoá	Cấu trúc	Chặn kiểu gì
1	<code>(hotel_id, iso_date)</code>	<code>set[str]</code> dạng <code>"hid YYYY-MM-DD"</code>	Chặn thô — an toàn nhất, cố tình over-flag
2	hash 120 ký tự đầu của <code>norm_text(review_text)</code>	<code>dict[hotel_id, set[str]]</code>	Chặn khớp nội dung kể cả khi ngày lệch
3	<code>hotel_id → gold_split</code>	<code>dict[str, str]</code>	Không phải để chặn, mà để gắn <code>gold_split</code> → dựng cohort T-unbiased

`is_gold_leak(hid, iso, text_pos, text_neg, keys_date, keys_text)` — **early-exit** nếu `hid` không phải hotel gold (đa số dòng thoát ở đây, nên hàm rẻ); sau đó thử khoá ngày, rồi thử khoá text trên **ba biến thể**: `text_pos`, `text_neg`, và `f"{text_pos} {text_neg}"` — vì gold có thể được nối từ cả hai ô.

Over-flag là thiết kế, không phải bug

Run thật: `flag_gold_leak = 18.475` > 8.796 review gold — vì khoá `(hotel, date)` cố tình thô (một hotel có thể có nhiều review cùng ngày). Đây là hướng an toàn: thả loại nhầm vài nghìn dòng khỏi training **còn hơn để một dòng test lọt vào**. Nếu `flag_gold_leak == 0` → **blocklist hổng, phải dừng lại** (đã xác nhận gold < pool nên con số 0 là bất khả thi).

● Khoá `(hotel_id, review_date)` hiện **KHÔNG** hoạt động đã đổi

`hamos-mabsa/metadata/reviews.jsonl` **không có trường** `review_date`, và `metadata/reviews_with_dates.jsonl` (đường ưu tiên của `_hamos_file`) không tồn tại trong bản checkout hiện tại. Đo trực tiếp: `blocklist: 0` khoá `(hotel,date)` · 3.399 hotel gold — tức **chỉ còn khoá hash text hoạt động**, không phải 2 lớp như mô tả ở trên.

Hệ quả đo được: chạy lại Module A trên môi trường hiện tại cho `flag_gold_leak = 1.466` thay vì 18.475 — **~17.000 dòng pool từng bị chặn nay lọt vào tập được train / self-train / infer**. Cohort không đổi (270/1.207/514/10.629) nên không có dấu hiệu nào khác.

Module A nay log **ERROR** và ghi `blocklist_date_keys` + `blocklist_date_key_active` vào `store_report.json`. **Kiểm `blocklist_date_key_active` trước mỗi run**. Muốn bật lại lớp này: bổ sung `metadata/reviews_with_dates.jsonl` vào repo `hamos-mabsa` rồi chạy lại Module A và mọi bước sau.

5.3 Tám bước xử lý (theo đúng thứ tự trong `build_store()`)

#	Bước	Logic cài đặt	Kết quả run thật
A1	Parse ngày	<code>U.parse_review_date</code> ; không parse được → loại dòng (<code>drop_no_date</code>)	184 dòng bị loại
A2	Cắt cửa sổ	<code>U.in_window(period)</code> — ngoài 2022-03...2025-02 → loại	12.602 dòng
A3	Dedup	Khoá <code>text_hash("hid name iso text_pos text_neg")</code> lưu trong set trong RAM	15.157 dòng (0,78%)
A4	Gold blocklist	Không loại dòng — gắn cờ <code>in_gold=true</code> . Việc bỏ qua thực hiện ở downstream (infer/selftrain/photos/audit)	18.475 dòng bị gắn cờ
A5	Ép kiểu <code>hotel_id</code>	<code>str(d.get("hotel_id") or "")</code> ở pool và <code>str(row["hotel_id"])</code> ở gold — khớp kiểu <code>str ↔ int</code>	—
A6	Cờ text-bearing	Tách <code>text_pos</code> / <code>text_neg</code> riêng (NFC), <code>has_text = bool(pos or neg)</code> , <code>n_words</code> = số từ của hai ô ghép lại	1.141.532 có text
A7	Language ID	CHƯA CÀI — docstring nêu là tùy chọn (cần <code>fasttext</code>); output không có trường <code>lang</code>	—

A8	Stratum + cohort	<code>U.make_stratum(country, state, n_words)</code> ghi thành list; cohort tính sau khi ghi xong từ <code>pool_count</code> + <code>hotel_split</code>	4 cohort, xem §2.7
----	------------------	--	--------------------

Ba điểm lệch giữa code và tài liệu phương pháp LỆCH

- **Không lọc dòng không có text.** Store giữ cả 780.129 dòng không text (chỉ score+title), chỉ gán cờ `has_text`. Việc lọc xảy ra tự nhiên ở `module_b_infer.units()` (chỉ sinh đơn vị khi ô text tồn tại). Nên `n_written` = 1.921.661 **không phải** số review có text.
- **Không có trường `lang`** (A7 chưa cài) và **không có `image_ids`** — thay bằng `photo_urls` (list URL). `method_table_q1.md §A.3` mô tả schema có hai trường này.
- **`hotel_id` giữ kiểu `str`** trong output (tài liệu ghi “đã ép kiểu int”). Mọi so sánh downstream cũng dùng str nên nhất quán.

5.4 Schema `outputs/store/reviews.jsonl.gz` (19 trường)

Trường	Kiểu	Nguồn / công thức	Ái dùng
<code>review_uid</code>	str	<code>f"{hotel_id}_{iso không gạch}_{chỉ số dòng đã đọc:08d}"</code> — ổn định theo thứ tự file, không ổn định nếu pool đổi	khóa join xuyên pipeline; tên file ảnh pool
<code>hotel_id</code>	str	ép str	cohort filter (B-infer, D)
<code>iso_date</code>	str	<code>YYYY-MM-DD</code>	blocklist, audit
<code>period</code>	str	<code>YYYY-MM</code>	trục thời gian của Module D
<code>text_pos</code> / <code>text_neg</code>	str null	NFC của <code>review_positive</code> / <code>review_negative</code>	hai đơn vị suy luận $\varphi=\text{POS}$ / $\varphi=\text{NEG}$
<code>score</code>	float null	<code>U.parse_score</code>	IPW propensity (C3); chỉ để validate, không bao giờ lọc
<code>stars_rating</code>	float null	passthrough	phân tầng bổ sung (chưa dùng)
<code>stratum</code>	list[str]	<code>[country_bloc, traveller]</code> (2 phần tử vì <code>USE_LENGTH_IN_STRATA=False</code>)	direct standardization của Module D
<code>country_bloc</code> · <code>traveller</code>	str	bản phẳng của <code>stratum</code> (tiện query)	phân tích phụ
<code>n_words</code>	int	số từ của <code>"{pos} {neg}"</code>	length_bin, feature bridge, IPW
<code>nights</code> · <code>room</code>	int null · str null	<code>parse_nights(d["date"])</code> · NFC	kiểm soát “đổi mix sản phẩm” (chưa dùng)
<code>has_text</code>	bool	<code>bool(text_pos or text_neg)</code>	mẫu D0
<code>has_photo</code>	bool	<code>bool(d["review_photo"])</code>	C2/C3 (propensity), tải ảnh
<code>photo_urls</code>	list[str]	<code>list(d["review_photo"].keys())</code>	<code>download_pool_photos.py</code> lấy URL đầu tiên
<code>in_gold</code>	bool	cờ blocklist	BLOCKLIST — không bao giờ train/self-train/infer trên dòng này
<code>gold_split</code>	str null	<code>train/dev/test</code> ở cấp hotel	dựng cohort T-unbiased

5.5 Kết quả run thật (`store_report.json` , 2026-08-21)

stats	Giá trị	Tổng hợp	Giá trị
<code>read</code>	1.949.604	<code>n_written</code>	1.921.661
<code>drop_no_date</code>	184	<code>n_hotels</code>	10.629
<code>drop_window</code>	12.602	cohort <code>A-dense</code>	270
<code>drop_dup</code>	15.157	cohort <code>B-anchor</code>	1.207
<code>has_text</code>	1.141.532	cohort <code>T-unbiased</code>	514
<code>no_text</code>	780.129	cohort <code>corpus</code>	10.629
<code>flag_gold_leak</code>	18.475 (1.466 trên môi trường hiện tại — khoá date rỗng, xem §5.2)	<code>window</code>	2022-03 → 2025-02

Kiểm tra: 1.949.604 – 184 – 12.602 – 15.157 = 1.921.661 ✓ (dòng bị gán cờ gold vẫn được ghi).

src/prism/module_b_data.py · python -m prism.module_b_data · ~10 s · không cần torch

6.1 Ngữ pháp target tuyến tính hoá

```
TASK_PREFIX = "extract quads: "
QUAD_OPEN   = "<quad>"    QUAD_CLOSE = "</quad>"    SEP = " | "    NULL = "NULL"

input : "extract quads: Nhân viên nhiệt tình"
target: "<quad> Nhân viên | SER_ATTITUDE | nhiệt tình | positive </quad>"

nhiều quad → nối bằng dấu cách:
target: "<quad> staff | SER_ATTITUDE | friendly | positive </quad> <quad> pool | AM_POOL | dirty | negative </quad>"

aspect/opinion implicit → token NULL:
target: "<quad> NULL | AM_WIFI | NULL | negative </quad>"
```

Thứ tự 4 trường trong một quad: **aspect_term | taxonomy_code | opinion_term | sentiment**. **aspect_category** không nằm trong target — nó được suy ra từ **CODE2CAT[code]** lúc parse (giảm độ dài target và loại một nguồn mâu thuẫn).

Term chứa ký tự phân cách phải được escape đã đổi

linearize nay đưa term qua **esc_term()**: `|` → `|`; `<quad>` / `</quad>` → dạng entity; **parse_linearized** gọi **unesc_term()** để khôi phục. Vì sao bắt buộc: **aspect_term** = "giá | chất lượng" sinh ra **5 trường**, và bản cũ yêu cầu đúng 4 nên **bỏ cả quad im lặng**. Hai đường thiết hại: lúc train, target vẫn chứa quad đó nên model được dạy sinh định dạng mà chính parser sẽ bỏ; lúc infer, mọi quad có `|` trong term bị mất → **recall lệch theo loại term** (rất thường trong review đa ngữ: "giá | chất lượng", "wifi|tv"), không lệch theo aspect nên **không xuất hiện ở bảng by_category** nào. **roundtrip_mismatch** có đếm, nhưng đó là bằng chứng cho gold hôm nay, không phải bảo vệ cho pool.

6.2 parse_linearized(s) — nghịch đảo, dùng cả lúc infer

Thuật toán **đã đổi**: `split(QUAD_OPEN)[1:]` → cắt tại `QUAD_CLOSE` → tách `|` → **neo vào hai trường xác định được**: **sentiment** là trường CUỐI, **taxonomy_code** là trường **duy nhất** thuộc **CODE2CAT**. Phần trước code = aspect, phần giữa code và sentiment = opinion, rồi **unesc_term()**. Nhờ vậy model tự sinh `|` thô vẫn parse được thay vì mất cả quad. Bỏ qua (im lặng) quad khi:

- < 4 trường (model sinh sai định dạng);
- không có **đúng một** trường thuộc **CODE2CAT** (không xác định được vị trí code);
- vị trí aspect hoặc opinion rỗng hẵn;
- code** ∉ **CODE2CAT** → **taxonomy hard-filter ngay tại parse**;
- sentiment** ∉ {positive, neutral, negative}.

Trả về dict 7 trường: **aspect_term · taxonomy_code · aspect_category · opinion_term · sentiment · aspect_implicit · opinion_implicit** (***_implicit** = trường đó bằng **NULL**).

Hard-filter tách khỏi score mềm

Taxonomy hợp lệ là **ràng buộc cứng** (quad có code lạ không tồn tại về mặt khái niệm) nên nó nằm ở parse, và lặp lại ở **apply_weights** dưới dạng **1[tax hợp lệ]**. Ngược lại provenance/reliability là **score mềm**, không bao giờ được biến thành bộ lọc.

6.3 Round-trip check & chronological probe (E1c)

Với mỗi split, code chạy lại **parse_linearized(linearize(quads))** và so số quad thu được với số quad gold **có code hợp lệ**; lệch → tăng **roundtrip_mismatch**. Smoke test **assert bằng 0**.

Chronological probe: ghép **train + dev** rồi cắt theo **review_date**: **chrono_train** ≤ 2024-06, **chrono_test** ≥ 2024-07. **Không dùng test gốc** để bảng F1 chuẩn vẫn nguyên vẹn.

6.4 Output & số liệu run thật (data_report.json)

File	Segment	Quad	Ghi chú
outputs/extract/train.t2t.jsonl	8.816	16.789	input train của B-train
outputs/extract/dev.t2t.jsonl	1.895	3.600	early-stop theo dev loss / dev F1
outputs/extract/test.t2t.jsonl	1.890	3.606	E1 — dùng một lần cuối
outputs/extract/chrono_train.t2t.jsonl	7.046	—	E1c: train lại từ đầu trên tập này
outputs/extract/chrono_test.t2t.jsonl	3.665	—	E1c: đo temporal generalization

16.789 + 3.600 + 3.606 = **23.995** ✓ đúng tổng quad gold (smoke test assert con số này).

Mỗi dòng ***.t2t.jsonl**: **instance_id · source_review_id · review_date · input · target · n_quads**.

§7 · Module B-train — Seed extractor mT5 GPU

`src/prism/module_b_train.py` · `python -m prism.module_b_train --model google/mt5-small --epochs 10`

7.1 Vì sao mT5

Gold đa ngữ **80,5% en · 16,2% vi**, còn pool thì ~30% có dấu tiếng Việt — **ngịch đảo phân phối**. Một backbone chỉ-tiếng-Anh sẽ sập trên phần pool tiếng Việt; một backbone chỉ-tiếng-Việt sẽ sập trên phần gold tiếng Anh. mT5 là lựa chọn tối thiểu-rủi ro; oversample gold tiếng Việt là hướng cải thiện đã ghi nhận (chưa cài).

7.2 Tham số CLI (mặc định trong code)

Cờ	Mặc định	Cơ sở chọn
<code>--model</code>	<code>google/mt5-base</code>	Kaggle thực tế dùng <code>mt5-small</code> (orchestrator mặc định) vì hạn mức VRAM T4
<code>--epochs</code>	10	self-train gọi lại với <code>--epochs 3</code>
<code>--batch</code> · <code>--grad-accum</code>	8 · 4	batch hiệu dụng 32; Kaggle dùng 4 · 8
<code>--lr</code>	3e-4	AdamW + linear schedule, warmup 6% tổng số bước, clip grad-norm 1,0
<code>--max-src</code>	160	segment mean 13 từ, max 147
<code>--max-tgt</code>	192	max 23 quad/segment
<code>--seed</code>	<code>RANDOM_SEED = 20260821</code>	seed cả <code>random</code> và <code>torch</code>
<code>--train-file</code> · <code>--dev-file</code>	<code>extract/train.t2t.jsonl</code> · <code>dev.t2t.jsonl</code>	đổi sang <code>chrono_train.t2t.jsonl</code> cho probe E1c
<code>--out</code>	<code>models/seed_extractor</code>	self-train ghi <code>models/selftrain_round<r></code>

Device tự chọn `cuda` → `mps` → `cpu`. Label padding được đặt `-100` để loại khỏi loss. **Keep-best theo dev loss**: chỉ lưu checkpoint khi dev loss giảm (kèm tokenizer), nên thư mục output luôn là epoch tốt nhất chứ không phải epoch cuối.

7.3 Output

```
models/seed_extractor/
├─ config.json generation_config.json model.safetensors # 2,1 GB với mt5-small (fp32)
├─ tokenizer.json tokenizer_config.json (+ spiece)
└─ training_log.json # {"args": {...}, "history": [...], "best_dev_loss": ..., "device": ...}
```

7.4 Log run thật (mt5-small · T4 · 10 epoch)

Epoch	1	2	3	4	5	6	7	8	9	10
train loss	5,867	0,408	0,315	0,270	0,241	0,222	0,208	0,196	0,188	0,183
dev loss	0,332	0,265	0,234	0,217	0,210	0,201	0,197	0,193	0,190	0,191 ↑
giây/epoch	395	399	397	397	397	397	399	397	398	397

best_dev_loss = 0,190413 tại epoch 9 → checkpoint lưu là epoch 9 (epoch 10 dev loss tăng nhẹ, đúng hành vi keep-best). Tổng ~66 phút. **Lưu ý**: dev loss thấp **không** thay cho F1 — phải chạy §8 để có số báo cáo.

§8 · Module B-eval — E1 / E1b / E1c GPU tùy chọn

`src/prism/module_b_eval.py` · không nằm trong chuỗi Kaggle bắt buộc, nhưng là **bảng số tham chiếu bắt buộc của paper**

8.1 Định nghĩa metric

`quad_key(q) = ( norm_text(aspect_term), taxonomy_code, norm_text(opinion_term), sentiment )`

Exact match = trùng **cả 4 thành phần** — cùng quy ước với `evaluation/quadruple_metrics.py` của HAMoS. `aspect_category` không tham gia (nó là hàm của code).

So khớp bằng **multiset** (`collections.Counter`) rồi lấy giao & → xử lý đúng trường hợp một instance có nhiều quad giống nhau. `prf(n_gold, n_pred, n_hit)` → {P, R, F1, gold, pred, hit}, làm tròn 4 chữ số.

8.2 Sáu chiều phân tích

Khoá report	Cấp lọc	Nhóm	Dùng để làm gì
<code>overall</code>	—	toàn test	E1 — số F1 chính
<code>by_language</code>	instance	<code>en · vi · other</code> (từ <code>languages[0]</code> của gold meta, có fallback lấy từ chính dòng split). Bản cũ so <code>== "other"</code> với mã ngôn ngữ thật (ko/zh/fr/?) nên bucket này luôn rỗng và mọi review không en/vi bị bỏ khỏi bảng . Nay report ghi kèm cờ <code>by_language_covers_overall</code> — bất biến: tổng <code>gold</code> của 3 bucket phải bằng <code>overall.gold</code>	E1b — dự kiến vi thấp hơn en rõ rệt

by_length	instance	L0/L1/L2 theo số từ của <code>g["text"]</code>	E1b — L2 thấp = xác nhận recall lệch theo độ dài (D-a)
by_polarity	instance	instance vào bucket s nếu có quad gold mang s	chỉ để tham khảo — bucket trộn lẫn quad khác cực
by_polarity_quad	quad	chỉ quad mang đúng s	số đo trực tiếp cho M3a (recall lệch theo cực tính)
by_category	quad	6 category	báo cáo per-category (per-code vô nghĩa với 17 code hiếm)

Output: `<out-prefix>_report.json` (mặc định `outputs/extract/eval_report.json`) + `<prefix>_preds.jsonl` (`{instance_id, quads}` cho phép audit tay). Sinh prediction bằng `generate(num_beams=4, max_length=--max-tgt)`, batch 16.

Cách đọc bảng E1

P3

- `by_language.vi` << `en` → đúng dự đoán; cân nhắc oversample vi.
- `by_length.L2` thấp → **số này đi thẳng vào paper** làm bằng chứng cho D-a.
- E1c tụt mạnh so với E1 → temporal generalization kém → **mọi kết luận drift phải hạ mức tự tin.**

S9 · Module B-selftrain — Teacher → Student GPU

`src/prism/module_b_selftrain.py` · `python -m prism.module_b_selftrain --rounds 2 --cohort B-anchor`

Vòng lặp self-training **có kiểm soát**: điều khác biệt với self-training thông thường không phải là vòng lặp, mà là (i) pseudo-label được chọn theo **hai** ngưỡng — model confidence và provenance posterior; (ii) hai guardrail dừng sớm dựa trên **phân phối lớp**, không chỉ dev F1.

9.1 Bốn bước mỗi vòng

#	Bước	Chi tiết cài đặt
1	Pseudo-label	<code>subprocess</code> gọi <code>prism.module_b_infer --ckpt <teacher> --cohort <cohort> --limit <infer-limit></code> . Dòng <code>in_gold</code> bị bỏ tự động bên trong infer. Kết quả ghi đề <code>extract/pool_quads.<cohort>.jsonl.gz</code>
2	Chọn pseudo	<code>pseudo_rows()</code> giữ quad có <code>conf_seq ≥ --tau</code> VÀ <code>p_posterior ≥ --tau-post</code> , gom theo <code>(review_uid, phi)</code> thành instance t2t (target = <code>linearize</code> các quad còn lại), shuffle theo <code>RANDOM_SEED + rnd</code> , cắt còn <code>max_n = --max-ratio × gold_train </code>
3	Train student	Ghi <code>extract/selftrain_round<r>.t2t.jsonl = gold_train + pseudo</code> ; gọi <code>module_b_train --model <teacher ckpt> --train-file <mixed> --out models/selftrain_round<r> --epochs 3</code> → tiếp tục từ teacher , không train lại từ base
4	Đo dev & dừng	Gọi <code>module_b_eval</code> trên <code>dev.t2t.jsonl</code> + gold <code>data/raw/dev.jsonl</code> → đọc <code>overall.F1</code> ; dừng nếu F1 giảm so với vòng trước

9.2 Hai guardrail (dấu hiệu error propagation)

Guardrail 1 — **lệch phân phối lớp**: dừng nếu `%negative(pseudo) - 0,154 > --max-skew (0,10)`.

`0,154 = gold_neg_rate` [đo trên gold] — nay **đo tại chỗ** bằng `gold_negative_rate(gold_train)` mỗi lần chạy **đã đổi**, không hardcoded, nên gold cập nhật thì guardrail tự theo. Pseudo lệch cực tính là dấu hiệu extractor tự khuếch đại bias của chính nó — nếu để chạy tiếp, kênh v sẽ trôi vì **công cụ**, không vì dữ liệu.

Guardrail 2 — **dev F1 không cải thiện**: self-train đo dev F1 của **teacher gốc** trước vòng 1 (ghi thành `round: 0` trong history) và chỉ nhận checkpoint mới khi nó **vượt mốc tốt nhất đã thấy** **đã đổi**. Bản cũ chỉ so với vòng ngay trước nên vòng 1 *luôn* được nhận, kể cả khi nó làm dev tụt so với seed — và `final_ckpt` có thể tệ hơn seed mà không log nào nói ra.

Cả hai guardrail đều **break** ra khỏi vòng lặp, và `final_ckpt` giữ ckpt của vòng cuối **đã pass**.

9.3 Tham số CLI

Cờ	Mặc định	Ý nghĩa
<code>--rounds</code>	2	số vòng tối đa
<code>--cohort</code>	<code>B-anchor</code>	pool để pseudo-label (1.207 hotel — đủ đa dạng, chưa quá to)
<code>--tau</code>	0,7	ngưỡng <code>conf_seq</code>
<code>--tau-post</code>	0,8	ngưỡng <code>p_posterior</code> — lọc mềm : quad <code>provenance_flip</code> vẫn được giữ nếu posterior đủ cao
<code>--max-ratio</code>	3,0	pseudo tối đa = $3 \times \text{gold train} = 26.448$ instance
<code>--max-skew</code>	0,10	guardrail 1
<code>--infer-limit</code>	200.000	số đơn vị suy luận tối đa mỗi vòng

9.4 Output

```
models/selftrain_round1/ , models/selftrain_round2/ ...
outputs/extract/selftrain_round<r>.t2t.jsonl      # tập trộn gold ∪ pseudo
outputs/extract/dev_round<r>.report.json          # eval từng vòng
outputs/extract/selftrain_history.json
{ "history": [{ "round": 1, "dev_f1": "...", "n_pseudo": "...", "neg_rate": "...", ... },
  "final_ckpt": "models/selftrain_round2",          # ← infer chính thức PHẢI dùng cái này
  "args": { ... } }
```

9.5 Cạm bẫy vận hành

Store là input của self-train, không phải sản phẩm của B

Bước infer nội bộ cần `outputs/store/reviews.jsonl.gz` + `hotel_cohorts.json`. Trên Kaggle, quên attach store → lỗi “thiếu hotel_cohorts.json”. Đây là output của Module A.

Ngân sách thời gian: `--infer-limit` mặc định 200k là quá lớn cho Kaggle

Notebook `04_selftrain` dùng `--limit 4000`; ghi chú trong repo: mặc định 200k mất **~58 h/vòng** trên T4 — vượt hạn mức phiên (~9–12 h). Tăng dần khi có thêm quota. `module_b_selftrain` nay nhận trực tiếp `--batch` / `--score-batch` và truyền xuống bước infer nội bộ; orchestrator không còn vá source lúc chạy (§17.4).

S10 · Module B-infer — Pool chính thức + provenance posterior GPU

`src/prism/module_b_infer.py` · `python -m prism.module_b_infer --ckpt models/selftrain_round<N> --cohort <cohort>` · **step nặng nhất** của pipeline

10.1 Đơn vị suy luận: mỗi review tối đa 2 đơn vị

`units()` đọc store theo dòng và sinh **một đơn vị cho mỗi ô text không rỗng**: (`text_pos`, `φ=POS`) và (`text_neg`, `φ=NEG`). Bỏ qua: dòng `in_gold=true` (blocklist) và hotel không thuộc cohort (`keep` rỗng = corpus = mọi hotel).

Vì sao gán ϕ theo từng đoạn chứ không theo review

Nguồn trường là thuộc tính của **đoạn văn bản**: cùng một review có đoạn nằm trong ô “điểm cộng” và đoạn nằm trong ô “chưa hài lòng”. Gán ϕ ở mức review sẽ trộn hai prior rất khác nhau (0,931 vs 0,641) và làm posterior mất ý nghĩa.

10.2 `conf_seq` — độ tin cậy chuỗi tính bằng teacher forcing

`conf_seq = exp( (1/|y|) ·  $\sum_i \log P(y_i | x, y_{<i})$  )` — trung bình log-prob token của **chính chuỗi đã sinh**, rồi lấy exp.

Cài trong `seq_conf()`: chạy lại một forward pass với `labels = chuỗi đã decode`, `log_softmax`, rồi `gather` đúng log-prob của token đã chọn và mask padding.

Vì sao không dùng `generate(output_scores=True) + compute_transition_scores`

Cách đó giữ logits **full-vocab (~250k của mT5)** cho **mọi** bước sinh → OOM ngay trên T4. Cách hiện tại chỉ giữ log-prob của token đã chọn và chunk theo **score-batch**. Đây là lý do tồn tại của `seq_conf / seq_logprobs` thay vì gọi API tiện lợi hơn. **đã đổi** Hai hàm nay là wrapper mỏng của **một** hàm `_score()` trả cả tổng và trung bình log-prob trong **cùng một lượt forward** — trước đây là hai bản sao gần như từng dòng, nghĩa là hai lần phải sửa khi đổi giới hạn token (nay lấy từ `config.MAX_SRC_TOKENS / MAX_TGT_TOKENS` thay vì hardcoded 160/192 ở 6 chỗ).

Rescore bị **truncate** làm posterior thoái hoá thành prior thuần **đã đổi**

Bước rescore dựng **toàn bộ** target cho mỗi biến thể sentiment, nên unit có `n` quad phải chấm **3n** chuỗi **mỗi chuỗi dài n quad** — chi phí **bậc hai** theo số quad (đây cũng là lý do `--score-batch` tách riêng khỏi `--batch`).

Với `MAX_TGT_TOKENS = 192`, unit nhiều quad (khoảng $\geq 9-10$ quad, tùy tokenizer) bị truncate mất **chính cái quad đang xét** → cả 3 biến thể tokenize **giống nhau** → log-prob bằng nhau → `p_model = (%,%,%)` → posterior thành **prior thuần**: sentiment do ϕ quyết định hoàn toàn và `provenance_flip` bật lung tung. Không exception nào, chỉ là số. Gold có segment tới **23 quad** nên đường này chậm được.

Nay phát hiện bằng `max(lp) - min(lp) < DEGENERATE_LP_EPS` và ghi hai cỡ vào từng dòng output: `p_model_exact` (rescore thật) và `p_model_truncated`. Cuối run log tỷ lệ. Module C/D vì thế lọc được các quad này.

10.3 `P_model(s|x)` — rescore 3 biến thể sentiment

Với **mỗi quad** đã sinh, tạo 3 target biến thể — **chỉ đổi sentiment của đúng quad đó**, giữ nguyên mọi quad khác — rồi tính **tổng log-prob** của từng biến thể (`seq_logprobs`) và softmax trên 3 giá trị đó:

$$P_{\text{model}}(s | x) = \text{softmax}_{s \in \{\text{pos, neu, neg}\}} \left(\sum_i \log P(y_i^{(s)} | x, y_{<i}^{(s)}) \right)$$

Chi phí: 3 forward pass **mỗi quad** (không phải mỗi review) — đây chính là lý do infer đắt. `--no-rescore` thay bằng xấp xỉ one-hot (**0,9 / 0,05 / 0,05**); khi đó “posterior” chỉ còn là hàm tất định của (`s` dự đoán, ϕ) và **l mất ý nghĩa** → chỉ dùng để debug.

10.4 Provenance posterior

$$P(s | x, \phi) \propto P_{\text{model}}(s | x)^\lambda \cdot P(s | \phi)^{1-\lambda} \quad (\lambda = 0,7)$$

Cài trong `apply_provenance()` dưới dạng log-linear rồi softmax ổn định số (trừ max trước khi `exp`): `post[s] =  $\lambda \cdot \log P_{\text{model}}[s] + (1-\lambda) \cdot \log \text{prior}[s]$` . Trả về (`s*`, `p_posterior`) với `s* = argmax` và `p_posterior` = xác suất chuẩn hoá của `s*`.

Nếu `s* ≠ sentiment_model` → gán cờ `provenance_flip = true`. **Quad bị lật vẫn được giữ** (không lọc) để audit — đây là nhóm tập trung mĩa mai, phủ định và lỗi extractor.

10.5 Vòng `flush()` — trình tự thực thi một batch

- `tok(TASK_PREFIX + text)` → enc (truncate 160)
- `model.generate(num_beams=4, max_length=192)` → chuỗi target
- `tok.decode(skip_special_tokens=True)` → `texts_out`
- `seq_conf(inputs, texts_out)` → `conf_seq` mỗi ĐƠN VỊ
- `parse_linearized(texts_out[i])` → danh sách quad (đã hard-filter taxonomy)
- nếu KHÔNG `--no-rescore`:
 - với mỗi (đơn vị bi, quad qi, sentiment s) → dựng target biến thể
 - `seq_logprobs`(tất cả biến thể, chunk theo `--score-batch`) → softmax → `p_model[(bi,qi)]`
- `apply_provenance(p_model, phi, lam)` → (`sentiment`, `p_posterior`)
- ghi 1 dòng JSON / quad vào `.jsonl.gz` (ghi streaming, không giữ trong RAM)

10.6 Schema `outputs/extract/pool_quads.<cohort>.jsonl.gz`

Nhóm	Trường	Ghi chú
Quad	<code>aspect_term · taxonomy_code · aspect_category · opinion_term · aspect_implicit · opinion_implicit</code>	từ <code>parse_linearized</code>
Ngữ cảnh	<code>text · review_uid · hotel_id · period · stratum · phi · score · has_photo · n_words</code>	copy từ store (theo đơn vị , nên <code>text</code> là ô text của đúng ọ đó)
Suy luận	<code>conf_seq</code> (4 số) · <code>sentiment_model</code> · <code>p_model{positive,neutral,negative}</code> · <code>sentiment</code> (= s* hậu nghiệm) · <code>p_posterior</code> · <code>provenance_flip</code>	Cẩn thận: <code>sentiment</code> là posterior , sentiment gốc của model nằm ở <code>sentiment_model</code> . Module D và C đều đọc <code>sentiment</code>

Chưa có `quad_uid` ở bước này — nó được thêm ở `module_c_reliability.apply_weights` (và dựng lại on-the-fly ở `fit_bridge` để join với mẫu audit).

10.7 Tham số & ngân sách

Cờ	Mặc định	Ghi chú
<code>--ckpt</code>	<code>models/seed_extractor</code>	Kết quả chính thức phải trả về tới <code>final_ckpt</code> của self-train
<code>--cohort</code>	<code>T-unbiased</code>	chạy cohort nhỏ trước, <code>corpus</code> sau cùng
<code>--batch</code>	32	Kaggle T4 dùng 2
<code>--score-batch</code>	48	batch cho rescore; Kaggle dùng 4
<code>--lam</code>	0,7	= <code>PROVENANCE_LAMBDA</code>
<code>--limit</code>	0	0 = không giới hạn; đếm theo đơn vị (không phải review)
<code>--no-rescore</code>	off	Không dùng cho kết quả paper (quy tắc bất biến §19.4)

Thứ tự khuyến nghị: `T-unbiased` (514 hotel) → `B-anchor` → `corpus` (~1,15M review có text — chạy qua đêm/nhiều phiên).

§11 · Ảnh pool & hai mẫu annotation người

11.1 scripts/download_pool_photos.py NET

IN	outputs/store/reviews.jsonl.gz
OUT	outputs/reliability/pool_images/<review_uid>.jpg + pool_image_index.json = {review_uid: đường dẫn TUYỆT ĐỐI}
Chọn review	has_photo=true ∧ in_gold=false ∧ thuộc cohort ∧ có photo_urls
Lấy ảnh nào	ảnh đầu tiên (photo_urls[0]), chỉ nhận scheme http/https
Cờ	--cohort (T-unbiased) · --limit 0 · --sleep 0.4 · --timeout 15
Resume	Ảnh đã có trên đĩa (size>0) được bỏ qua nhưng vẫn vào index; index được checkpoint mỗi 200 ảnh mới
Lỗi	Đếm n_errr , log 20 lỗi đầu, không dừng chạy

Đường dẫn tuyệt đối → photos và các stage C phải chạy cùng session

pool_image_index.json lưu path tuyệt đối (/kaggle/working/...). Attach lại ở notebook khác thì path lệch và apply_verifier sẽ bỏ qua toàn bộ ảnh (im lạng: quad chỉ không có v_image) → bridge báo “chưa có v_image”.

11.2 make_audit_samples.py — hai mẫu tách biệt

Mẫu	Tạo khi nào	File	Phân tầng	Người annotate điền	Dùng cho
D0	ngay sau Module A	reliability/d0_sample_300.jsonl	length_bin (3) × có/không text_neg (2) = 6 ô, CAP 200/ô, 50 mẫu/ô	gold_quads : toàn bộ quad của review (văn bản đầy đủ, không cắt)	D-a: bảng recall p(sentiment, length_bin) → outputs/drift/recall_table.json
AUDIT	sau khi có pool_quads	reliability/audit_sample_300.jsonl	conf_band (lo<0,6 · mid<0,85 · hi) × phi (2) × provenance_flip (2) = ≤12 ô, CAP 300/ô, 25 mẫu/ô	correct : 0 1 cho từng quad	Module C3: fit bridge + Spearman go/no-go

Không được trộn hai mẫu P1

D0 đo recall của extractor (phải annotate đầy đủ văn bản); AUDIT đo precision của pseudo-quad (chấm đúng/sai từng quad đã sinh). Dùng cùng một mẫu cho cả hai là circular: bridge sẽ được hiệu chỉnh trên chính tập dùng để đo recall.

Reservoir sampling chuẩn (Algorithm R) cho từng ô: giữ CAP phần tử đầu, sau đó phần tử thứ k thay một slot ngẫu nhiên với xác suất CAP/k. Bản cũ dùng xác suất cố định → mẫu lệch về các dòng đến sau trong file (và file pool sắp xếp theo thời gian ⇒ mẫu lệch theo thời gian). D0 loại in_gold và các dòng không có text.

src/prism/module_c_reliability.py · --stage {train_verifier, apply_verifier, bridge, apply} — 4 stage đúng thứ tự này

12.1 Vấn đề module này giải

Mọi tín hiệu text-based — model confidence, cross-model agreement, field provenance — **chia sẻ cùng một nguồn lỗi**. Hai model train trên cùng gold sẽ cùng sai một kiểu; agreement cao **không** chứng minh đúng. Đây là lỗi hổng circularity mà không tín hiệu text nào bịt được. Ảnh là một **sensor khác**: nó không chia sẻ nguồn lỗi đó. Đóng góp không phải “dùng ảnh để extract tốt hơn” (multimodal ABSA đã làm) mà là **dùng ảnh để ước lượng độ tin cậy pseudo-label, rồi suy rộng ra ngoài phạm vi có ảnh** — 6,24% review có ảnh hiệu chỉnh cho 93,76% còn lại.

12.2 Giới hạn phạm vi (bắt buộc khai báo — chính nó làm claim đáng tin)

Giới hạn	Số đo	Hệ quả
Đa số review chỉ có 1 ảnh	95,6% (8.412/8.796)	Grounding ở mức review , không phải mức quad
Một ảnh phải “gánh” nhiều aspect	42,6%	review 1-ảnh có quad trái >1 taxonomy code → ảnh không xác thực được toàn bộ quad
Cực tính lẫn lộn dưới 1 ảnh	13,3%	Ảnh khó xác thực s ở mức quad
Review nhiều ảnh gán đúng ảnh khác nhau	349/384 = 90,9%	Cỡ mẫu quá nhỏ để làm claim quad-level

→ **Phạm vi hợp lệ**: ảnh xác thực **kênh category** và cực tính thô ở **mức review**. **Không** xác thực span (aspect_term, opinion_term). Vì vậy verifier được học ở mức **REVIEW × CATEGORY** — đây là phạm vi duy nhất mà alignment cho phép.

12.3 Ngưỡng go/no-go — chốt trước khi chạy (chống HARKing)

GO_NOGO = {"verifier_auc_min": 0.70, "verifier_delta_auc_min": 0.05, "bridge_spearman_min": 0.30}

#	Kiểm tra	Ngưỡng	Vì sao ngưỡng đó
1	AUC của V trên gold test	≥ 0,70	Sàn “ảnh có tín hiệu”
2	ΔAUC = AUC(V) – AUC(baseline không ảnh)	≥ 0,05	Chống rò rỉ category-prior : one-hot category tự dự đoán được y vì FACILITY có mặt ở hầu hết review. AUC tuyệt đối cao mà không vượt baseline nghĩa là ảnh không đóng góp gì — go/no-go phải dựa trên delta
3	Spearman(Ĥ, human audit 300 quad)	≥ 0,30	Bridge phải tương quan với đánh giá người, không chỉ với chính verifier

NO-GO không chặn pipeline

Trượt bất kỳ ngưỡng nào → bỏ Module C; **apply_weights** tự **fallback w = conf_seq** (nên chạy kèm temperature scaling) và bài lùi về A+B+D — vẫn đủ tải một bài Q1 (IP&M/KBS), chỉ mất mũi nhọn Information Fusion. Vì vậy **chạy C sớm** để biết scope, dừng để cuối.

12.4 C1 · **train_verifier** GPU

Dựng dataset (**build_verifier_dataset**)

Bước	Chi tiết
Ảnh của review	<code>GOLD_IMAGES</code> → <code>img_of[source_review_id] = [local_path...]</code>
Category của review	<code>GOLD_QUADS</code> → <code>rid = quad_id.rsplit("_Q",1)[0]</code> → <code>cats_of[rid] = {aspect_category...}</code>
Nhân	Positive: (ảnh, c, y=1) với mọi c ∈ <code>cats_of[rid]</code> . Negative: sample <code>min(pos , neg_pool)</code> category không xuất hiện trong review → cân bằng ~1:1
Split	Lấy từ <code>{train,dev,test}.jsonl</code> theo <code>source_review_id</code> → hotel-disjoint được thừa hưởng ; review không tra được split → mặc định <code>"train"</code>
Seed	<code>random.Random(RANDOM_SEED)</code> — chọn negative tái lập được

Mô hình

Encoder	<code>open_clip ViT-B-32 / laion2b_s34b_b79k</code> , ĐỒNG BẰNG (A8: ghi <code>encoder_version</code> vào report để chống “model version drift”)
Text prompt	6 câu tiếng Anh mô tả 6 category (ví dụ FACILITY = “a photo of a hotel room, building, bathroom or view”), embed một lần rồi L2-normalize
Đặc trưng	<code>x = [cos_sim(img, text_c)] + one_hot(c)</code> → 7 chiều
Head	<code>LogisticRegression(max_iter=1000)</code> , train trên <code>{train, dev}</code> , test trên <code>{test}</code>
Baseline	Cùng head nhưng bỏ cột sim (<code>X[:, 1:]</code>) → chỉ one-hot category
Cache	Embed mỗi ảnh một lần , lưu <code>reliability/image_feats.npy</code> + <code>image_feats_index.json</code> ; ảnh mở lỗi → bỏ qua (không crash)

Output `verifier_report.json`

```
{ "auc_test": ..., "auc_no_image_baseline": ..., "delta_auc": ...,
  "auc_by_category": {"FACILITY": ..., ...}, # AUC dùng RIÊNG cos-sim trong từng category
  "n_train": ..., "n_test": ...,
  "go_nogo": "GO" | "NO-GO", "thresholds": {...},
  "encoder_version": "open_clip/ViT-B-32/laion2b_s34b_b79k" }
+ reliability/verifier.pkl = {"head", "model_name", "pretrained", "prompts"}
```

`auc_by_category` quan trọng khi đọc kết quả: trong **một** category, one-hot là hằng số nên AUC ở đó phản ánh **thuần tín hiệu ảnh**. Category có AUC $\approx 0,5$ nghĩa là prompt cho category đó vô dụng (dự kiến: LOYALTY, BRANDING — “ảnh mà khách quay lại sẽ chụp” không có nội dung thị giác rõ ràng).

12.5 C2 · `apply_verifier` GPU

IN	<code>--quads = extract/pool_quads.<cohort>.jsonl.gz; --image-index = reliability/pool_image_index.json; verifier.pkl</code>
OUT	<code>--out = reliability/pool_quads_vimg.<cohort>.jsonl.gz</code> — mọi quad được ghi lại, quad có ảnh được thêm trường <code>v_image</code> (4 số)
Logic	Nếu <code>review_uid ∈ index</code> : embed ảnh (cache theo uid, lỗi → cache <code>None</code>), tính <code>sim</code> với text-embed của category của quad đó, dựng lại <code>[sim] + one_hot</code> , chạy <code>head.predict_proba(...)[0,1]</code>
Log	<code>"v_image cho N/M quad"</code> — N=0 nghĩa là index sai path hoặc chưa tải ảnh

Lưu ý ghép nối: stage `bridge` phải đọc **file vimg này** (không phải `pool_quads` gốc) vì nó cần `v_image`; còn stage `apply` đọc **pool_quads gốc** + `bridge.pkl`. Orchestrator Kaggle đã nối đúng như vậy (§17.3).

12.6 C3 · `bridge` — hạt nhân của đóng góp CPU

$$\hat{f}(q) = g(\text{conf_seq}, p_{\text{posterior}}, 1[\phi=\text{POS}], \log(1+n_{\text{words}}), 1[\text{provenance_flip}]) \approx \Pr[V(\text{ảnh}, c_q) = 1]$$

`g` là `LogisticRegression` fit trên tập quad có `v_image`, nhãn nhị phân `y = 1[v_image ≥ 0.5]`. Vì mọi feature **chỉ dùng text**, `f` áp được cho **toàn bộ** quad — kể cả ~1,03M review không ảnh.

Hiệu chỉnh covariate shift bằng IPW (bỏ là sai)

Nhóm pool có text	n	Số tử TB	Score TB
Có ảnh	114.988	56,7	8,90
Không ảnh	1.035.427	36,5	8,68

Gold **100%** có ảnh, pool chỉ **6,24%** → covariate shift nặng. Cài đặt: propensity `LogisticRegression` trên `[log(1+n_words), score (điểm 8,7 nếu null)] → P(has_photo)`; trọng số `w_ipw = 1 / clip(p, 0.02, 1.0)` (clip chặn trọng số nhỏ). Bridge được fit **hai bản**: có IPW (`bridge`) và không IPW (`bridge_noipw`) — để báo cáo có/không hiệu chỉnh. **Bản dùng cho `apply` là bản có IPW**.

Đối chiếu human audit

Nếu `--audit` tồn tại: đọc `{quad_uid: correct}` (bỏ dòng `correct=null`), dựng lại khoá bằng `U.quad_uid(q)` cho **mọi** quad, join, và nếu ≥ 30 **cặp** thì tính `spearmanr(f, correct) → ghi audit_spearman · audit_p · audit_n · go_nogo`. Ít hơn 30 cặp → **không** ghi verdict (im lặng) — đây là dấu hiệu mẫu audit chưa được annotate hoặc khoá không khớp.

```
reliability/bridge.pkl = {"bridge", "bridge_noipw", "propensity"}
reliability/bridge_report.json = {"n_with_image", "n_total", "coef", "feature_names",
                                  ["audit_spearman", "audit_p", "audit_n", "go_nogo"]}
```

12.7 C4 · `apply` — gắn `w` cho mọi quad CPU

$$w_q = 1[\text{taxonomy hợp lệ}] \cdot 1[\text{span hợp lệ}] \cdot \hat{f}(q)$$

Trạng thái cài đặt: code hiện tính `hard = 1.0 if taxonomy_code in CODE2CAT else 0.0` — **chỉ có thành phần taxonomy**; `1[span hợp lệ]` chưa được cài (span của pool quad chưa được kiểm tra lại với text). **LỆCH**

IN	<code>--quads = pool_quads gốc (không phải vimg) + bridge.pkl</code> nếu có
OUT	<code>reliability/quads_weighted.jsonl.gz</code> — thêm <code>w</code> (4 số) và <code>quad_uid</code> (truy vết row-level)
Fallback	đã đổi <code>apply_weights</code> chỉ dùng <code>bridge</code> khi <code>verdict == "GO"</code> (verdict lưu <i>trong</i> pickle), và kiểm parity đặc trưng (<code>feature_names + n_features_in_</code>) trước khi predict — lệch thì <code>SystemExit</code> chứ không sinh <code>w</code> rác. NO-GO hoặc thiếu <code>bridge.pkl → f = conf_seq thô</code> (KHÔNG temperature-scale — repo chưa implement), và mỗi dòng ra mang <code>w_source</code> để Module D truy vết. Bản cũ chỉ kiểm <code>bp.exists()</code> nên bridge NO-GO vẫn được dùng cho <code>w</code> của toàn corpus và hợp đồng “NO-GO → fallback” không bao giờ chạy. Pipeline vẫn chạy tới D

src/prism/module_d_drift.py · python -m prism.module_d_drift --quads outputs/reliability/quads_weighted.jsonl.gz --cohort corpus --level taxonomy_code --n-boot 1000

13.1 Hai độ lệch, khác nhau về bản chất

	Độ lệch	Bảng chứng đo được	Bản chất	Trạng thái cài
D-a	Recall lệch theo cực tính × độ dài	Gold chỉ phủ 26% văn bản gốc (5.355 cặp ghép: 23,6 vs 92,7 từ, token overlap 0,97); phủ ô positive 42% nhưng negative chỉ 25%	Lỗi đo của công cụ	STUB — xem \$13.9
D-b	Đổi thành phần reviewer	VN 89,7%→16,1%; %có text 42,1→63,4%; độ dài 27,5→40,8 từ; hotel/tháng 549→8.535	Đổi thành phần tổng thể	đã cài + kiểm chứng

Gộp hai thứ này làm một là **sai khái niệm** — và chính sự phân biệt đó là một đóng góp phương pháp cho temporal opinion mining.

13.2 load_quads — gom về 3 cấu trúc

```
cell[(period, stratum)][aspect] = Σ w # kênh π - CHỈ quad complaint
vcell[(period, stratum)][aspect][sentiment] = Σ w # kênh v - MỌI quad
reviews[(period, stratum)] = [(review_uid, {aspect: w}), ...] # cho bootstrap
```

Quyết định	Cài đặt
Lọc cohort	<code>cohort_hotels</code> rỗng = không lọc (corpus). Đọc từ <code>store/hotel_cohorts.json</code> ; file không tồn tại + cohort ≠ corpus → <code>SystemExit</code> với hướng dẫn chạy Module A
Mức aspect	<code>--level taxonomy_code</code> : code không thuộc <code>CODES_REPORTABLE</code> → tự gộp lên category cha (<code>CODE2CAT</code>). <code>--level aspect_category</code> : dùng thẳng category
Trọng số	<code>w = q.get("w", q.get("conf_seq", 1.0))</code> → chấp nhận cả <code>quads_weighted</code> (có <code>w</code>) và <code>pool_quads</code> (fallback <code>conf_seq</code>). <code>w ≤ 0</code> → bỏ quad
Định nghĩa “lượt phân nân”	<code>q["phi"] == "NEG" or q["sentiment"] == "negative"</code> — hợp của hai điều kiện: quad nằm trong ô “chưa hài lòng” <i>hoặc</i> có cực tính negative sau posterior

13.3 Kênh π — shares_by_period + shrinkage

Với mỗi kỳ t: tính **share gộp s0** của kỳ (mọi stratum cộng lại), rồi:

raw: $\pi_{a,t} = \text{pooled}_{a,t} / \Sigma_a \text{pooled}_{a,t}$
adj (direct standardization + shrinkage):
 $\hat{\pi}_{a,t} \propto \Sigma_g \pi_g^{\text{ref}} \cdot (n_{a,t,g} + m \cdot s0_{a,t}) / (n_{t,g} + m)$, `m = --min-stratum` (30)
 π_g^{ref} = tỷ trọng của stratum g **gộp toàn 36 kỳ** (`reference_composition`) — một thành phần tham chiếu **cố định**. Ô rỗng đóng góp đúng share gộp của kỳ (trung tính) thay vì bị loại. Cuối cùng chuẩn hoá lại về tổng 1.

Câu hỏi mà bước này trả lời: *nếu thành phần khách không đổi thì cơ cấu phân nân có đổi không?* Độ phủ strata đo được: trung bình **99,2%**, thấp nhất 74,2% → hiệu chỉnh gần như không mất dữ liệu. Chuỗi đưa vào phân tích trend là $\text{clr}(\pi_t)_a$ (§4.5).

13.4 Kênh v — valence_by_period

raw: $v_{a,t} = \Sigma_g w_{\text{neg}} / \Sigma_g w_{\text{all}}$ (tỷ lệ gộp của kỳ)
adj: $\hat{v}_{a,t} = [\Sigma_g \pi_g^{\text{ref}} \cdot (w_{\text{neg},g} + m_v \cdot v0)] / [\Sigma_g \pi_g^{\text{ref}} \cdot (w_{\text{all},g} + m_v)]$, `m_v = --min-valence-w` (10)
`v0` = tỷ lệ gộp của kỳ (prior Beta). Tham chiếu của kênh v được tính riêng (`reference_composition_valence`, dựa trên **mọi** quad) chứ không dùng lại tham chiếu của kênh π (chỉ complaint). Kỳ không có quad nào của aspect → **None** (bỏ khỏi chuỗi, không nội suy).

13.5 Máy trend

Hàm	Công thức / logic	Ghi chú
<code>deseason(series, periods)</code>	$y'_t = y_t - (\text{mean}(\text{tháng của } t) - \text{mean}(\text{toàn chuỗi}))$	Khử hiệu ứng tháng-trong-năm bằng trung bình theo tháng. Bắt buộc: biên độ mùa của FAC_CLIMATE/AM_POOL lớn gấp ~4× aspect khác
<code>ols_trend(series)</code>	OLS trên index kỳ; trả (<code>slope × 12</code> , <code>t = b/se</code>)	slope quy về mỗi năm ; se dùng residual với df = n-2. <code>n ≤ 2</code> → <code>se = 0</code> → <code>t = 0</code>
<code>best_split(series)</code>	$c^* = \text{argmax}_c \text{Welch-t}(y_{1:c}, y_{c+1:T}) $ với <code>c ∈ [4, T-4]</code>	Yêu cầu tối thiểu 4 điểm mỗi phía → không bắt changepoint ở hai biên
<code>permutation_p(...)</code>	Xáo thứ tự thời gian của chuỗi (giữ nguyên tập giá trị), tính lại <code>best_split(deseason(...))</code> , <code>p = (hits + 1)/(n_perm + 1)</code>	Null riêng cho từng chuỗi — ngưỡng khác nhau rõ rệt giữa các aspect. Cộng 1 ở tử/mẫu → p không bao giờ bằng 0

<code>channel_stats(...)</code>	Bỏ điểm None ; <12 điểm → trả None (chuỗi không đủ dài để kết luận)	Trả <code>{slope_per_year, t_stat, changepoint, welch_t, p_perm, p_perm_changepoint}</code> — hai kiểm định permutation riêng: p_perm cho thống kê TREND (khớp với <code>slope_per_year / t_stat</code> được báo cáo và với verdict) và p_perm_changepoint cho bước nhảy. đã đổi Trước đây chỉ permute thống kê changepoint rồi báo p đó cạnh slope OLS, nên significant_after_fdr thực chất là “có bước nhảy” trong khi verdict lại gate trên dấu slope — aspect trôi đơn điệu mà không có bước nhảy bị gọi là “phẳng”. BH-FDR nay chạy trên thống kê trend
---------------------------------	---	--

13.6 Bootstrap CI (kênh π -adj)

Resample **review trong từng ô** (kỳ $\times$ **stratum**) — có hoàn lại, giữ đúng số review của ô — rồi dựng lại **cell_b**, tính lại share adj, deseason, OLS. Percentile 2,5% / 97,5% → **adj.slope_ci95**.

Một bản resample dùng chung cho mọi aspect

Vòng bootstrap nằm **ngoài** vòng per-aspect: mỗi lượt tạo **một** bản resample rồi lấy slope cho tất cả aspect từ chính bản đó. Vừa nhanh gấp $|\text{aspects}|$ lần, vừa **đúng hơn** — các aspect chia sẻ cùng phương án resample nên tương quan giữa các share của một composition được bảo toàn (đây là dữ liệu compositional, không phải các chuỗi độc lập).

13.7 Hai lưới BH-FDR & năm nhãn verdict

Lưới	Thành phần	Vai trò
adj (chính)	π -adj U v-adj trên mọi aspect	Kết quả chính thức của paper
raw (riêng)	π -raw U v-raw	Chỉ để so like-for-like trong injection E3a — không phải kết quả chính

Mỗi entry được ghi **p_fdr** + **significant_after_fdr**. Verdict được tính riêng cho kênh π (**verdict**) và kênh v (**verdict_val**):

Nhãn	Điều kiện	Đọc thể nào
XU HƯỚNG THẬT	adj sig $\wedge$ raw sig $\wedge$ cùng dấu slope	Xu hướng tồn tại và không phải artifact thành phần
GIÁ (do thành phần)	raw sig $\wedge$ $\neg$ adj sig	Phân tích thô sẽ báo cáo sai — đây là ca Module D loại bỏ
BỊ CHE, lộ ra sau hiệu chỉnh	adj sig $\wedge$ $\neg$ raw sig	Tín hiệu thật bị thành phần che — ca Module D phát hiện thêm
ĐẢO DẤU sau hiệu chỉnh	adj sig $\wedge$ raw sig $\wedge$ trái dấu	Kết luận đảo chiều hoàn toàn (ví dụ AM_FOOD trong probe)
phẳng	còn lại	Không kết luận

13.8 Schema output `outputs/drift/drift_results.<cohort>.<level>.json`

```
{
  "config": { ...vars(args)... },
  "periods": ["2022-03", ...],      # trục của nu_*_series (từ vcell)
  "pi_periods": ["2022-03", ...],  # trục của pi_*_series (từ cell - chỉ kỳ có complaint)
  "n_significant_after_fdr": 4,     # đếm theo kênh  $\pi$ -adj
  "n_significant_valence_fdr": 3,   # đếm theo kênh v-adj
  "test_statistic": "ols_trend_t (permutation, null riêng từng chuỗi)",
  "recall_adjusted": false,         # true CHỈ khi D-a đã implement và áp (§13.2)
  "results": [
    { "aspect": "AM_FOOD", "level": "taxonomy_code", "cohort": "corpus",
      "raw": { "slope_per_year", "t_stat", "changepoint", "welch_t",
               "p_perm",                # kiểm định TREND - khớp t_stat, dùng cho BH-FDR
               "p_perm_changepoint",    # kiểm định BƯỚC NHẢY - báo cáo riêng
               "p_fdr", "significant_after_fdr" },
      "adj": { ...như trên..., "slope_ci95": [lo, hi] },      # CI chỉ có khi --n-boot > 0
      "val_raw": { ...hoặc null nếu <12 điểm...,
        "val_adj": { ... },
      "pi_raw_series": [...], "pi_adj_series": [...],      # share (không phải clr)
      "nu_raw_series": [...], "nu_adj_series": [...],      # có thể chứa null
      "verdict": "ĐẢO DẤU sau hiệu chỉnh", "verdict_val": "phẳng" },
    ...
  ]
}
```

Log cuối cùng in một dòng/aspect, sắp theo $|t|$ của π -adj giảm dần — dùng để đọc nhanh mà không cần mở JSON.

13.9 D-a: trạng thái stub **CHƯA HOÀN THIỆN**

Bước hiệu chỉnh recall hiện chỉ log, chưa áp vào ước lượng

Code kiểm tra `outputs/drift/recall_table.json`; nếu có thì log "D-a: áp dụng `recall_table.json`" và đọc file — **nhưng không nhân $1/\rho$ vào cell/vcell**. Nếu không có thì log "D-a: BỎ QUA". Công thức đích: $\tilde{N}_{a,t,s} = \sum_{\ell} N_{a,t,s,\ell} / \rho^{\text{rec}}(s, \ell)$ với ρ ước lượng từ **tập D0** (~300 review annotate đầy đủ). **Phải hoàn thiện trước run kết quả cuối**, và cần bảng recall thật trước đó (chặn bởi annotation).

13.10 Tham số CLI

Cờ	Mặc định	Ghi chú
<code>--quads</code>	bắt buộc	<code>quads_weighted.jsonl.gz</code> hoặc <code>pool_quads.*.jsonl.gz</code>
<code>--level</code>	<code>taxonomy_code</code>	hoặc <code>aspect_category</code>
<code>--cohort</code>	<code>corpus</code>	4 lựa chọn; cần <code>hotel_cohorts.json</code> nếu $\neq$ corpus
<code>--min-stratum</code>	30	cường độ shrinkage kênh π
<code>--min-valence-w</code>	10	cường độ shrinkage kênh v
<code>--n-perm</code>	1000	injection dùng 300 cho tiết kiệm; smoke test dùng 100
<code>--n-boot</code>	0	0 = tắt. Kết quả cuối phải truyền <code>--n-boot 1000</code>
<code>--fdr</code>	0,05	—
<code>--seed</code>	20260821	dùng cho cả permutation và bootstrap
<code>--out</code>	auto	<code>drift_results.<cohort>.<level>.json</code> ; injection dùng cờ này để không ghi đè kết quả chính

§14 · Injection & negative control (E3/E4) CPU

`src/prism/eval_injection.py` · `python -m prism.eval_injection --quads <file> --test {composition,valence,shuffle}` · đây là bảng evaluation trung tâm của paper

Vì **không có drift ground truth**, ta tạo drift/không-drift **có kiểm soát** trên chính dữ liệu thật rồi kiểm tra pipeline trả lời đúng. Mỗi injection **chỉ đối đúng một thứ**; kết quả ghi vào file riêng (`drift_results.inj_*.json` + `injection_<test>.json`) và **không ghi đè** kết quả corpus chính.

14.1 E3a · composition — hiệu chỉnh phải im lặng

Bước	Cài đặt
1. Dựng nền null	<code>inject_shuffle</code> đã đổi : xáo nhối <code>period</code> giữa các <code>review_uid</code> rồi phát xuống mọi quad của cùng review (giữ multiset kỳ mức review và mọi nội dung). Bản cũ shuffle list period rồi <code>zip</code> theo từng quad , nên hai quad của cùng một review lạc sang hai tháng khác nhau — null chứa những review không tồn tại, và với E4 thì phá tương quan trong review → phương sai chuỗi bị hạ → FPR thực nghiệm bị ước lượng THẤP , negative control PASS quá dễ
2. Tiêm đổi thành phần	<code>inject_composition</code> : theo kỳ, giữ quad với xác suất <code>keep_p</code> — VN đi từ 1,0 → 0,2; non-VN 0,2 → 1,0 (mô phỏng đúng chiều VN 89,7→16,1%). Resample-with-drop, không sửa nội dung quad nào
3. Chạy drift	<code>--level taxonomy_code --cohort corpus --n-perm 300</code> , ghi ra <code>drift_results.inj_composition.json</code>
4. Chấm	Đếm aspect significant sau FDR ở lưới raw (π -raw U v-raw) và lưới adj (π -adj U v-adj)

Verdict	Điều kiện	Đọc thế nào
PASS	<code>n_adj == 0</code> $\wedge$ <code>n_raw</code> ≥ 1	Đúng như thiết kế: thô báo drift giả, adj im lặng
FAIL	<code>n_adj > 0</code>	Hiệu chỉnh hỏng — adj báo drift trên nền null
INCONCLUSIVE	cả hai = 0	Injection quá yếu để raw bắt được (mix aspect giữa các strata quá giống nhau) — không nói được gì về adj

Hai chi tiết thiết kế dễ làm sai

- **Phải xáo timestamp trước**. Dữ liệu thật có thể chứa drift thật; tiêm thẳng lên đó rồi đòi adj im lặng là yêu cầu vô lý.
- **Cùng một máy suy diễn cho cả raw và adj** (permutation + FDR). So “t-threshold cho raw” với “FDR cho adj” là so táo với cam.

14.2 E3c · valence — kênh v phải bắt được

Injection	<code>inject_valence(quads, aspect, δ, t0, rng)</code> : sau <code>t0</code> , lật $\delta\%$ quad <code>positive</code> → <code>negative</code> của một aspect. Giữ nguyên φ — injection đối đúng một biến
Grid	$\delta \in \{2, 5, 10, 20\}\%$ · mặc định <code>--aspect AM_FOOD --t0 2023-09</code>
Chấm	Với từng δ : <code>val_adj</code> phải significant sau FDR và changepoint nằm trong ± 3 kỳ của <code>t0</code> (<code>within3</code>). Kênh π -adj báo phụ (quad lật vẫn vào kênh π qua điều kiện <code>sentiment == "negative"</code>)
Output	<code>{"aspect", "t0", "detections": {"2%": {valence_detected_within_3, valence_changepoint, valence_p_fdr, prevalence_detected_within_3, prevalence_p_fdr}, ...}}</code>

Đây là chỗ đo **minimum detectable effect**: δ nhỏ nhất mà v-adj còn bắt được là con số mô tả độ nhạy của phương pháp.

14.3 E4 · shuffle — negative control

Injection	Xáo timestamp toàn bộ, lặp <code>--repeats</code> lần (mặc định 5) với seed <code>RANDOM_SEED + i</code>
Đo	FPR mỗi lượt = (số test significant) / (số test có <code>p_fdr</code>) trên lưới π -adj U v-adj; lấy trung bình
Verdict	PASS nếu mean FPR $\leq 2\alpha$ (= 0,10)
Vì sao phải lặp	Một lần chạy với ~14 aspect có độ phân giải quá thô để ước lượng FPR (bước nhảy nhỏ nhất $\approx 1/28$)

Smoke test chạy sẵn E4 trên dữ liệu tổng hợp; run thật cho **FPR = 0,0 trên 5 lượt lặp**.

14.4 Probe độc lập

`scripts/probe_complaint_composition.py` — CACT keyword-probe: đếm lượt nhắc bằng **lexicon sinh từ** `aspect_term gold + seed thủ công` (13 code), chạy trên 701.776 review có nội dung “chưa hài lòng”, rồi thực hiện đúng chuỗi standardization → CLR → deseason → OLS. Không bắt buộc trong pipeline; nó là **bảng chứng khả thi** có trước và độc lập với extractor (§15). Output: `outputs/probe/outputs_cact_probe.pkl` .

\$15 · Bảng chứng đã có — probe CACT

Proxy keyword, 701.776 review “chưa hài lòng”, 36 tháng, **chưa dùng ASQP**. Slope trên thang CLR mỗi năm, đã khử mùa vụ; $|t| > 2,5$ coi là có ý nghĩa. ⚠️ Bảng này **chưa hiệu chỉnh đa so sánh** — dùng để chứng minh khả thi, **chưa phải con số để trích vào bài**.

15.1 Hiệu chỉnh thành phần đối kết luận ở 8/13 aspect

Aspect	THỎ slope	t	ĐÃ HIỆU CHỈNH	t	Kết luận
AM_FOOD	+0,018	+1,4	-0,053	-7,4	bị che + đảo dấu
SER_SUPPORT	+0,042	+3,6	-0,009	-1,7	giả
FAC_ENV	-0,085	-4,4	-0,008	-1,1	giả — 91% artifact
AM_TRANSPORT	-0,031	-4,8	-0,006	-0,6	giả
SER_ATTITUDE	+0,014	+2,8	+0,001	+0,1	giả
FAC_VIEW_LOCATION	+0,031	+3,1	+0,023	+1,6	giả
FAC_BATH	-0,002	-0,4	+0,022	+3,4	bị che
AM_ROOM_UTIL	-0,025	-1,6	+0,023	+2,6	bị che
AM_WIFI	-0,121	-6,9	-0,109	-6,7	thật, bền vững
AM_POOL	+0,126	+6,0	+0,037	+3,5	thật (nhỏ hơn 71%)
FAC_BUILDING	+0,048	+4,3	+0,016	+2,8	thật (nhỏ hơn 67%)
FAC_ROOM	+0,009	+2,6	+0,013	+3,1	thật
FAC_CLIMATE	-0,024	-0,9	+0,053	+2,1	đảo dấu, vẫn dưới ngưỡng

5 xu hướng giả bị loại · 3 xu hướng bị che được phát hiện · 4 xu hướng thật được xác nhận.

Ca sạch nhất — **AM_FOOD** : chuỗi thô đi từ 7,67% (2022-03) lên ~10% ($t = +1,4$ — “không có gì”); chuỗi đã hiệu chỉnh đi từ **12,74% xuống ~9,6%** ($t = -7,4$). Đầu 2022 reviewer là 89,7% người Việt và nhóm này phản nản về đồ ăn ở tỷ trọng thấp hơn nhóm khác → kéo con số thô xuống một cách giả tạo. Phân tích thô sẽ (i) báo cáo sai rằng “*phản nản về không gian/tiếng ồn giảm mạnh*” và (ii) **bỏ sót** việc phản nản về đồ ăn thật ra đang **giảm**.

15.2 Xác thực ngoại vi: khôi phục đúng quy luật vật lý

Không được cung cấp bất kỳ thông tin nào về mùa, nhưng profile tháng-trong-năm ra đúng như vật lý đòi hỏi:

Aspect	Đỉnh	Đáy	Đỉnh/đáy	Hợp lý?
FAC_CLIMATE (điều hoà)	T6 — 2,54%	T1 — 1,15%	2,22×	✔️ đỉnh giữa hè
AM_POOL (hồ bơi)	T8 — 4,38%	T12 — 2,53%	1,73×	✔️ đỉnh mùa bơi
FAC_BATH (nước nóng)	T12 — 9,45%	T8 — 7,89%	1,20×	✔️ đỉnh mùa lạnh
FAC_ROOM	T11 — 22,84%	T6 — 21,67%	1,05×	✔️ đúng là không có mùa
AM_WIFI	T3 — 1,73%	T1 — 1,36%	1,27×	trung tính

Đây là **bảng chứng validity mạnh nhất có được khi không có gold drift**: phương pháp tái tạo đúng chu kỳ đã biết trước, và cho **null phẳng đúng chỗ cần phẳng** (FAC_ROOM 1,05× — nếu FAC_ROOM cũng “có mùa” thì đó là dấu hiệu pipeline hỏng). Nó cũng chứng minh khử mùa vụ là **bắt buộc**: biên độ mùa của điều hoà/hồ bơi lớn gấp ~4× các aspect khác — không khử thì một cửa sổ so sánh đặt lệch mùa sẽ tự sinh ra “drift”.

15.3 ASQP thêm được gì so với keyword probe

1. **Trọng số tin cậy**: đếm Σw_q thay vì đếm 1.
2. **Kênh v thật**: keyword chỉ biết “đoạn này nằm trong ô negative”; ASQP cho cực tính ở mức quad → xử lý được 21,7% ca nội dung tích cực nằm trong ô negative.
3. **Kênh opinion $E_{a,t}$** (centroid embedding của opinion): phát hiện “*vẫn chê, nhưng lý do đổi*” — keyword mù hoàn toàn với việc này. chưa cải
4. **Subcategory + aspect implicit**: keyword không bắt được aspect ẩn (3,3% gold).

Hệ quả về thứ tự công việc: khung phân tích thời gian **đã kiểm chứng xong** — nên xây extractor để *nâng cấp* một pipeline đang chạy, thay vì xây extractor rồi mới hy vọng phân phân tích hoạt động.

S16 · Giao thức đánh giá, ablation, baseline, rủi ro

16.1 Bảng thí nghiệm E1–E11

ID	Thí nghiệm	Đo cái gì	Có gold?	Ưu tiên	Trạng thái
E1	Extraction F1 exact-match quad trên test 1.890 segment	Chất lượng extractor	✓	P1	cần chạy
E1b	F1 tách theo ngôn ngữ / độ dài / cực tính (mức quad)	Nguồn của D-a	✓	P1	cần chạy
E1c	Chronological probe (train $\leq 2024-06$ / test $\geq 2024-07$)	Temporal generalization	✓	P1	dữ liệu đã có
E2	AUC verifier + $\Delta\text{AUC} + \hat{r} \leftrightarrow$ human audit + IPW	Module C sống hay chết	bán phần	go/no-go	chưa chạy
E3a	Injection composition trên nền null	adj im lặng $\leftrightarrow$ raw báo	tổng hợp	lỗi	đã cài
E3b	Injection length/recall (kéo dài văn bản theo xu hướng thật 27,5→40,8 từ)	Như trên, cho D-a	tổng hợp	lỗi	chưa cài
E3c	Injection valence $\delta \in \{2,5,10,20\}\%$	Minimum detectable effect	tổng hợp	P1	đã cài
E4	Negative control — xáo timestamp $\times 5$ seed	FPR thực nghiệm sau FDR	✓	P1	FPR = 0,0
E5	Cohort train-hotel vs test-hotel (2.381 vs 514)	Mức lạc quan hoá của pseudo-label	✓	P2	cohort đã có
E6	Bootstrap stability (resample trong strata)	CI + độ bền changepoint	✓	P2	đã cài
E7	Cross-model consistency (đổi backbone/seed)	Spearman giữa 2 bảng xếp hạng drift	✓	P2	chưa
E8	Rating external check (held-out, dùng một lần)	Spearman kỳ vọng 0,3–0,7, chốt trước	weak	P2	chưa
E9	Seasonality sanity check	CLIMATE 2,22 $\times$ · POOL 1,73 $\times$ · ROOM 1,05 $\times$	vật lý	xong	PASS
E10	Event probe quanh 15/03/2022 (VN mở cửa du lịch)	Bảng chứng bổ trợ	Δ trùng confounder	P3	chưa
E11	Human temporal micro-benchmark (~400 review, mẫu tách biệt)	Chiều + độ lớn drift	✓	P2	chưa

16.2 Ablation bắt buộc

Ablation	Chứng minh điều gì	Trạng thái
– composition adjustment (D-b)	Giá trị lỗi của Module D	đã có: 8/13 aspect đối kết luận
– recall adjustment (D-a)	Xu hướng giả do độ dài trôi	chặn bởi D0
– CLR (hồi quy thô trên share)	Tương quan âm giả do ràng buộc simplex	dễ chạy
– deseasonalization	Biên độ mùa CLIMATE/POOL gấp $\sim 4\times$	đã có bằng chứng
– cross-modal reliability (C)	Giá trị của Module C	sau E2
– provenance prior (B)	Giá trị của weak supervision đặc thù	dễ chạy ($\lambda=1$)
– reliability weighting ($w \equiv 1$)	Uncertainty-aware aggregation có đáng công không	dễ chạy
– self-training (chỉ gold)	Giá trị của scaling	—
Strata 20 ô vs 60 ô (<code>USE_LENGTH_IN_STRATA=True</code>)	Robustness của hiệu chỉnh	1 cờ config
Tháng vs quý	Độ nhạy với lựa chọn thiết kế	—

16.3 Baseline phải thắng

Nhóm	Baseline	Vì sao bắt buộc
Extraction	Generative ASQP fine-tune, không self-training	số tham chiếu
	Self-training vanilla (chỉ lọc confidence)	baseline chính cần thắng — chứng minh giá trị của provenance
	LLM zero-shot & few-shot	reviewer 2026 chắc chắn sẽ hỏi
	Pipeline extract-classify 2 giai đoạn; keyword/lexicon	sàn dưới
Drift	Document-level rating trend (không dùng ABSA)	quan trọng nhất — không thắng thì bài mất lý do tồn tại
	Chính phương pháp nhưng KHÔNG hiệu chỉnh	bảng chứng trực tiếp cho đóng góp D-b
	Moving average · EWMA · CUSUM · Page-Hinkley	chuẩn drift detection
	PELT / Binary segmentation (<code>ruptures</code>)	chuẩn changepoint
	JS / Wasserstein giữa các kỳ; diachronic embedding shift	baseline cho kênh phân phối & semantic

16.4 Rủi ro & giảm thiểu

#	Rủi ro	Mức	Giảm thiểu (ở đâu trong code)	Trạng thái
R1	Composition drift nhằm thành quality drift	P1	D-b (<code>shares_by_period</code> / <code>valence_by_period</code> adjust) + E3a	đã giải quyết & chứng minh
R2	Leakage gold $\subset$ pool	P1	A4 blacklist + skip <code>in_gold</code> ở infer/selftrain/photos/D0	đã cài, 18.475 cỡ
R3	Recall lệch theo độ dài sinh drift giả	P1	D-a + E3b	chặn bởi D0
R4	Circular validation	P1	Quy tắc một-lần-một-phía : <code>review_score</code> chỉ validate, không bao giờ lọc pseudo-label	quy tắc §19.4
R5	Module C thất bại	P2	Go/no-go §12.3 + fallback <code>conf_seq</code>	đã cài fallback
R6	Covariate shift ảnh (gold 100% vs pool 6,24%)	P2	IPW <code>has_photo</code> + báo cáo bản <code>bridge_noipw</code>	đã cài
R7	Survivorship bias (hotel/tháng 549→8.535)	P2	Cohort A/B lọc theo mật độ; báo cáo độ nhạy	cohort đã có
R8	17/31 code không đủ mẫu	P2	Tự gộp lên category trong <code>load_quads</code>	đã quyết
R9	Extractor yếu trên tiếng Việt	P2	mT5 + <code>by_language</code> trong E1b; oversample vi	oversample chưa cài
R10	Rating tương quan quá cao → bài mất lý do tồn tại	P3	E8 chốt ngưỡng 0,3–0,7 trước khi chạy; case study aspect ngược chiều rating	chưa
R11	Lỗi rơi phủ định trong gold (<code>H10933639_R00009</code> : gold “Nhân viên nhiệt tình”/positive – gốc “nhân viên không nhiệt tình”)	P3	A9 audit 73 ca (5 ca kiểm thủ công: 4 đúng, 1 sai)	cần chốt tần suất

16.5 Tám giới hạn phải khai báo trong paper

1. Grounding ảnh ở **mức review**, không phải mức quad (95,6% single-image; 42,6% trái > 1 code).
2. Gold **100% có ảnh** vs pool 6,24% → covariate shift, đã hiệu chỉnh nhưng không triệt tiêu.
3. Gold chỉ phủ **26%** văn bản gốc; D-a giảm nhẹ nhưng không xoá được hệ quả.
4. **Không có drift ground truth** — bằng chứng đến từ injection + hội tụ nhiều tín hiệu, không phải một con số.
5. Một nền tảng (Booking.com), một quốc gia → khái quát hoá hạn chế.
6. **Không có `reviewer_id`** → không loại được reviewer lặp.
7. Span 2022-02→2025-03 **không chứa cú sốc COVID** → thiếu một nguồn validation ngoại sinh mạnh.
8. Implicit aspect chỉ 3,3%, implicit opinion 1,2% → **không** làm claim về implicit.

16.6 Vì sao không gộp drift thành một điểm số

Đã cân nhắc và **loại** công thức tổng hợp $D = \alpha \cdot D_{\text{sent}} + \beta \cdot D_{\text{sem}} + \gamma \cdot D_{\text{vol}}$: (i) thứ nguyên không tương thích ($JS \in [0,1]$, $|\Delta v| \in [0,2]$, $\text{cosine} \in [0,2]$, log-ratio không chặn); (ii) α, β, γ **không thể hiệu chỉnh** vì không có gold drift → hyperparameter tùy ý; (iii) gộp lại là **vứt bỏ** đúng điểm mạnh của bài — khả năng *phân biệt* các loại drift. Thay bằng: các chiều tách rời, mỗi chiều một p-value riêng, phân loại theo **bảng chữ ký** (prevalence / valence / semantic → sentiment drift · opinion drift · prevalence drift · emerging issue).

S17 · Orchestrator Kaggle — `scripts/kaggle_pipeline.py`

Gộp toàn bộ `src/prism` thành một `entrypoint`; triết lý: mỗi `cell` = 1 `step` để lỗi `step` nào chỉ hỏng `step` đó.

17.1 Danh sách `step` & `sentinel`

```
PIPELINE = ["store", "data", "train", "selftrain", "infer", "photos",  
            "c_verifier", "c_apply_verifier", "c_bridge", "c_apply", "drift"]  
STEPS     = PIPELINE + ["injection", "all"]      # injection KHÔNG nằm trong 'all'
```

Với `--step all`: chạy tuần tự và **BỎ QUA** `step` đã có output, dựa vào `sentinel()`; `--force` để chạy lại tất cả. Lỗi ở một `step` → in `"step X LỖI"` rồi dừng chuỗi (`step` đã xong sẽ tự skip ở lần chạy sau).

Step	File sentinel (có = coi như đã xong)
store	outputs/store/reviews.jsonl.gz
data	outputs/extract/train.t2t.jsonl
train	models/seed_extractor/model.safetensors
selftrain	outputs/extract/selftrain_history.json
infer	outputs/extract/pool_quads.<cohort>.jsonl.gz
photos	outputs/reliability/pool_image_index.json
c_verifier	outputs/reliability/verifier.pkl
c_apply_verifier	outputs/reliability/pool_quads_vimg.<cohort>.jsonl.gz
c_bridge	outputs/reliability/bridge.pkl
c_apply	outputs/reliability/quads_weighted.jsonl.gz
drift	outputs/drift/drift_results.<cohort>.<level>.json

17.2 Cơ chế tìm & stage input

Hàm	Logic	Rủi ro
<code>clone_repo</code>	Ưu tiên repo có sẵn tại <code>--repo-dir</code> ; nếu không, quét <code>/kaggle/input working/**/src/prism/config.py</code> ; cuối cùng mới <code>git clone --depth 1</code>	Không internet + không attach repo → lỗi rõ ràng
<code>find_one(name, roots)</code>	Glob đệ quy <code><root>/**/<name></code> khắp <code>/kaggle/input</code> rồi <code>/kaggle/working</code> ; lấy hit đầu tiên	Trùng tên file giữa nhiều dataset → lấy sai bản
<code>find_ckpt(roots)</code>	Thư mục HF hợp lệ = có <code>model.safetensors</code> + <code>config.json</code> ; xếp hạng theo tuple <code>("selftrain" ∉ path, "module-b-train" ∉ path, "seed_extractor" ∉ path, len(path))</code>	Ưu tiên có thể chọn sai khi attach nhiều ckpt → luôn truyền <code>--ckpt</code> tay
<code>find_hamos(roots)</code>	Tìm <code>**/annotations/quads.jsonl</code> rồi lùi 1-2 cấp để khớp <code>config._hamos_file</code> ; fallback tìm thư mục tên <code>hamos-mabsa</code>	Không thấy → <code>step data / c_verifier</code> báo lỗi rõ
<code>_safe_copy</code>	Bỏ qua khi nguồn trùng đích (đã stage từ <code>step</code> trước dưới <code>/kaggle/working</code>) → tránh <code>shutil.SameFileError</code>	—
<code>prepare_store</code>	Bắt buộc <code>hotel_cohorts.json</code> ; nhận <code>reviews.jsonl.gz</code> hoặc tự nén <code>reviews.jsonl</code> → <code>.gz</code>	—

Env được set trước khi gọi module con: `PRISM_TABSA_ROOT=<repo>` · `PRISM_WORK_DIR=/kaggle/working/outputs` · `PRISM_MODEL_DIR=/kaggle/working/models` · `PYTHONPATH=<repo>/src` · `PRISM_HAMOS_ROOT` (nếu tìm thấy) · `TOKENIZERS_PARALLELISM=false` · `PYTORCH_CUDA_ALLOC_CONF=expandable_segments=True`.

17.3 Ghép nối Module C trong orchestrator (đã xử lý sẵn)

```
c_apply_verifier : --quads outputs/extract/pool_quads.<cohort>.jsonl.gz  
                  --image-index outputs/reliability/pool_image_index.json  
                  --out outputs/reliability/pool_quads_vimg.<cohort>.jsonl.gz  
c_bridge         : --quads outputs/reliability/pool_quads_vimg.<cohort>.jsonl.gz  # ← BẮT BUỘC file vimg  
                  --audit outputs/reliability/audit_sample_300.jsonl           # MỘT tên duy nhất (config.HUMAN_AUDIT_NAME)  
c_apply          : --quads outputs/extract/pool_quads.<cohort>.jsonl.gz          # ← pool_quads GỐC  
                  --out outputs/reliability/quads_weighted.jsonl.gz  
drift            : ưu tiên quads_weighted; không có thì fallback pool_quads (w = conf_seq)
```

17.4 Không còn bản vá runtime **đã đổi**

Orchestrator **không còn quyền ghi vào `src/`**. Hai hàm `patch_infer_bug` và `patch_selftrain_batch` đã bị **xoá**.

Hàm cũ	Vi sao bỏ	Thay bằng
--------	-----------	-----------

patch_infer_bug	Needle <code>s_post != q["sentiment_model"]</code> không còn tồn tại trong source, nên hàm là no-op hoàn toàn — code chết nhưng vẫn có quyền ghi đè file	Không cần gì: <code>module_b_infer</code> vốn đã đúng (dict literal dựng <i>trước</i> <code>q.update</code>)
patch_selftrain_batch	So khớp một needle 2 dòng kèm đúng 24 space thật lè . Reformat / đổi tên biến → không khớp → không patch, không cảnh báo → self-train gọi infer ở mặc định 32/48 → OOM trên T4 <i>sau khi</i> đã train xong vòng 1	<code>module_b_selftrain</code> nay có cờ thật <code>--batch</code> / <code>--score-batch</code> , orchestrator truyền thẳng ở step <code>selftrain</code>

17.5 Chuỗi notebook (`kaggle_notebooks/`)

Notebook	Step	Máy	Attach (ngoài input gốc)
01_store	store	CPU	raw pool + splits + gold ABSA
02_data	data	CPU	hamos-mabsa + splits
03_train	train	GPU	output 02 (t2t)
04_selftrain	selftrain	GPU	output 03 (ckpt) + t2t + <code>dev.jsonl</code> + store
05_infer	infer	GPU	output 04 (ckpt student) + store
06_photos	photos	CPU + NET	store
07_c_verifier	c_verifier	GPU	hamos-mabsa (+ <code>pip install open_clip_torch</code> , cần Internet lần đầu để tải weight CLIP)
08_c_apply_verifier	c_apply_verifier	GPU	output 05 + 06 + 07
09_c_bridge	c_bridge	CPU	output 08 (vimg)
10_c_apply	c_apply	CPU	output 05 + 09 (<code>bridge.pkl</code>)
11_drift	drift	CPU	output 10 (<code>quads_weighted</code>)
00_full_pipeline	all	GPU + NET	tất cả — tự bỏ qua step đã có output

```
# Cell setup (đầu mỗi notebook)
!rm -rf /kaggle/working/Prism
!git clone --depth 1 https://github.com/hotuyen21pt/Prism.git /kaggle/working/Prism
P = "/kaggle/working/Prism/prism/scripts/kaggle_pipeline.py"

# mỗi cell 1 step
!python {P} --step train
!python {P} --step selftrain --cohort B-anchor --infer-batch 2 --infer-score-batch 4 --limit 4000
!python {P} --step infer --cohort T-unbiased --batch 2 --score-batch 4 \
    --ckpt /kaggle/input/notebooks/<user>/pipeline-selftrain/models/selftrain_round2
!python {P} --step photos --cohort T-unbiased
!python {P} --step c_verifier
!python {P} --step c_apply_verifier --cohort T-unbiased
!python {P} --step c_bridge --cohort T-unbiased
!python {P} --step c_apply --cohort T-unbiased
!python {P} --step drift --cohort corpus

# HOẶC chạy hết 1 cell (tự skip step đã có output)
!python {P} --step all --cohort T-unbiased # thêm --force để chạy lại tất cả
!python {P} --step all --cohort T-unbiased --limit 100 --epochs 3 --rounds 1 # chạy thử nhanh
```

Sau mỗi notebook GPU: bấm **Save Version (Save & Run All)** để output được giữ; attach tạm trong editor **không** lưu vào bản commit.

17.6 Cạm bẫy vận hành đã gặp thật

- 1 • infer** báo thiếu `hotel_cohorts.json`

Chưa attach output store (Module A) vào notebook infer. Attach dataset store **và** Save Version.
- 2 • Auto-detect checkpoint** chọn sai

Khi attach nhiều checkpoint, `find_ckpt()` có thể ưu tiên nhầm thư mục `seed_extractor` thay vì `selftrain_round2`. **Luôn truyền `--ckpt` tay tới student cuối.**
- 3 • photos** → `c_*` **phải cùng session**

`pool_image_index.json` chứa đường dẫn tuyệt đối (\$11.1).
- 4 • Không có hamos-mabsa** trên Kaggle

Bỏ Module C, chạy thẳng `--step drift --cohort corpus` — tự fallback dùng `pool_quads` với `w = conf_seq`.

5 · Ngân sách phiên

Kaggle giới hạn ~9–12 h GPU/phiên; `--step all` full thường không đủ → tách notebook 01→11. Bước `train`, `data`, `store` thường chạy local rồi upload output làm dataset.

18.1 Phân tầng phụ thuộc

Nhóm	Module cần	Cài gì
stdlib thuần	Module A · Module D · eval_injection · make_audit_samples · smoke_test · unit test · Module B-data	không cần gì (Python ≥3.9)
Numeric	Module C (bridge, propensity)	numpy · scipy · scikit-learn
Deep learning	B-train · B-eval · B-infer · B-selftrain	torch · transformers · sentencepiece · accelerate
Vision	C1/C2 verifier	open_clip_torch · pillow · torchvision
Tùy chọn	langid thật (A7) · kênh semantic E _{a,t} · tiện ích thống kê	fasttext-wheel · sentence-transformers · statsmodels

```
# Windows PowerShell
py -3.9 -m venv .venv; .\.venv\Scripts\Activate.ps1
python -m pip install -U pip; python -m pip install -r requirements.txt
$env:PRISM_HAMOS_ROOT='C:\path\to\hamos-mabsa'
$env:PYTHONPATH='src'

python -m unittest discover tests      # ~0,15s · 61 test · không cần dữ liệu/torch
python -m prism.smoke_test             # ~2 phút trên dữ liệu thật - PHẢI PASS
```

18.2 Docker

File	Base	Cài	Service / CMD
Dockerfile	python:3.12-slim	không cài gì (A + B-data chỉ dùng stdlib) → build vài giây	module-a · module-b-data
Dockerfile.torch	python:3.12-slim	torch bản CPU-only (nhẹ hơn ~1 GB so với CUDA) + transformers sentencepiece accelerate	module-b-infer --cohort T-unbiased

```
# Bắt buộc set trước khi chạy compose
$env:PRISM_DATA = "C:\...\prism\data\raw"      # phải chứa pool + splits + hotel_absa_labeled.jsonl
docker compose run --rm module-a
docker compose run --rm module-b-data
# kết quả ghi ra ./outputs/ trên host; data/raw mount read-only
```

Cả hai image set PYTHONPATH=/app/src và PRISM_TABSA_ROOT=/app tường minh (không dựa vào suy luận parents[2]). Muốn dùng GPU: đổi index-url của torch sang bản cu121 và thêm --gpus all.

18.3 Ba biến môi trường hay quên

Env	Khi nào phải set	Triệu chứng nếu quên
PYTHONPATH=src	Mọi lệnh local (repo dùng src-layout, chưa pip install -e)	ModuleNotFoundError: prism
PRISM_HAMOS_ROOT	hamos-mabsa không nằm cạnh repo	Smoke test dừng ở bước kiểm tra path; B-data/C1 báo thiếu gold
PRISM_WORK_DIR / PRISM_MODEL_DIR	Chạy trên Kaggle hoặc muốn tách output khỏi repo	Ghi vào <repo>/outputs (thường vẫn OK, nhưng trên Kaggle sẽ mất khi kết phiên)

§19 · Test, bất biến, trạng thái

19.1 Unit test — 61 test, stdlib, ~0,15 s

File	Test	Bất biến được bảo vệ
tests/test_utils.py	15	parse ngày (3 định dạng + ngày/tháng không hợp lệ) · parse score dấu phẩy · country_bloc 4 nhánh · length_bin biên · 36 kỳ trong cửa sổ · BH-FDR khớp ví dụ chuẩn + giữ thứ tự input + rỗng · Welch-t guard $n < 2$ và $\text{var} = 0$ · clr tổng = 0 + clr chịu được share = 0 · quad_uid ổn định & nhạy với opinion
tests/test_module_b_data.py	5	Round-trip quad explicit · implicit NULL cả aspect & opinion · giữ thứ tự nhiều quad · loại code lạ và sentiment lạ · chuỗi rỗng/không có quad/thiếu trường → trả list rỗng
tests/test_module_d.py	12	1 stratum: adj $\equiv$ raw · standardization xoá được composition shift (dựng ví dụ VN 75%→25%) · ô vắng đóng góp share gộp (shrinkage) thay vì tạo bước nhảy · prior rất lớn → shrink về gộp · v raw = tỷ lệ gộp · v adj = trung bình theo tham chiếu · aspect vắng → None · deseason xoá sạch mùa vụ thuần · OLS khôi phục đúng slope $\times 12$ · best_split tìm đúng vị trí bước nhảy · chuỗi <12 điểm → None · chuỗi phẳng → p_perm > 0,05 (nay là p của kiểm định trend)
tests/test_injection.py	5	valence injection chỉ đổi sentiment, không dụng cụ · không đổi gì trước t0 / aspect khác · shuffle bảo toàn multiset kỳ và mọi nội dung · composition chỉ drop chứ không sửa quad (kiểm bằng id()) · tỷ trọng VN giảm theo thời gian $\geq 0,3$
tests/test_regressions.py mới	24	Test chống hồi quy cho các lỗi đã sửa — mỗi cái khoá một lỗi mà trước đây <i>không test nào</i> bắt được: · term chứa hoặc </quad> round-trip được , và model tự sinh thô vẫn parse được (neo vào code + sentiment) — 7 test · inject_shuffle : mọi quad cùng review ở cùng một kỳ · multiset period mức review được bảo toàn · nội dung nguyên vẹn · tái lập với cùng seed — 4 test · parity quad_uid giữa make_audit_samples và module_c (khoá join của go/no-go) — 2 test · parity đặc trưng bridge fit ↔ apply + score=0.0 KHÔNG bị biến thành 8,7 — 3 test · find_ckpt chọn round LỚN NHẤT và ưu tiên selftrain_history.json · không còn hàm patch source · sentinel từ chối file rỗng — 5 test · COHORT_DEFS đủ khoá & Module A không còn literal ngưỡng — 2 test · ảnh HTML/JSON bị từ chối, magic bytes JPEG/PNG/WebP được nhận — 2 test

19.2 Smoke test — 5 kiểm tra trên dữ liệu thật

Smoke test nay ghi vào thư mục RIÊNG **đã đổi**

Mọi output của smoke nằm dưới **outputs/drift/smoke/**. Bản cũ gọi **module_d_drift** **không truyền --out**, nên ghi đè đúng **outputs/drift/drift_results.corpus.taxonmy_code.json** — file kết quả chính thức của cohort corpus — bằng 50k pseudo-quad keyword tổng hợp, cùng tên cùng schema, không cảnh báo. **eval_injection** nay có **--out-dir** cho cùng lý do.

#	Kiểm tra	Assert
1	Đường dẫn	Tồn tại POOL_JSONL · GOLD_QUADS · GOLD_META · train.jsonl
2	Blocklist	Chạy trên 20k dòng pool đầu; len(hotel_split) == 3399 (đúng số hotel gold — kỳ vọng đọc từ tests/expected_counts.json đã đổi , gold cập nhật thì sửa file đó chứ không sửa assert); log số leak bắt được (148/20k ở run thật)
3	Module B-data	train+dev+test quads == 23995 $\wedge$ roundtrip_mismatch == 0
4	Module D	Dựng pseudo-quad bằng keyword (5 code) từ 50k review pool, chạy drift --n-perm 100 ; assert ≥ 4 aspect, log số kỳ (36)
5	Negative control	Chạy eval_injection --test shuffle ; log FPR + verdict (FPR 0,0 → PASS)

Nếu smoke test fail thì dừng, không chạy tiếp bất cứ thứ gì.

19.3 Đường găng chuẩn

1. unit test + smoke test	✅ PASS
2. module_a_store	✅ ĐÃ CHẠY (1.921.661 review · 18.475 gold-leak flagged)
3. module_b_data	✅ ĐÃ CHẠY (23.995 quad · chrono 7.046/3.665)
3b. make_audit_samples (D0)	✅ ĐÃ CHẠY (300 review · 6 ô strata) → GIAO ANNOTATE NGAY (chặn D-a · ~1,5 tuần người)
4. module_b_train	✅ seed ĐÃ TRAIN (mt5-small · dev loss 0,1904)
5. module_b_eval (+ chrono probe E1c)	❑ chốt bằng F1 tham chiếu ← việc tiếp theo
6. module_c train_verifier	❑ GO/NO-GO theo ΔAUC — chạy SỚM để biết scope
7. module_b_infer T-unbiased → B-anchor → corpus	(giữ rescore BẬT)
7b. make_audit_samples --quads ... (AUDIT 300)	→ GIAO ANNOTATE
8. download_pool_photos → c apply_verifier → bridge → apply	
9. module_d_drift (corpus + T-unbiased, --n-boot 1000)	
10. eval_injection composition + valence + shuffle --repeats 5	
11. (tuỳ chọn) module_b_selftrain rồi lặp 7-10 với checkpoint mới	

19.4 Bày quy tắc bất biến — vi phạm = kết quả vô hiệu

- 1 · Không bao giờ train/self-train/infer trên dòng `in_gold=true`

Blocklist ở Module A + skip vô điều kiện ở `module_b_infer.units()` . Đừng bỏ blocklist khi refactor Module A “cho nhanh”.
- 2 · `review_score` chỉ dùng validate (E8), không bao giờ làm feature lọc pseudo-label

Đã đo: polarity field là nhãn sentiment gần-xác-định và `review_score` không độc lập với nó (pos-only score TB 9,52 · neg-only 5,33). Dùng cả hai cùng phía = circular validation.
- 3 · Không dùng temporal consistency làm bộ lọc

Nó xoá chính tín hiệu drift cần đo.
- 4 · Số chính thức = cột `adj` / `val_adj` sau FDR; luôn báo cáo kèm cột raw

Cột raw là bằng chứng minh bạch cho đóng góp D-b, không phải kết quả.
- 5 · Ngưỡng go/no-go (AUC 0,70 · ΔAUC 0,05 · Spearman 0,30) và ngưỡng E8 (0,3–0,7) chốt trước khi chạy

Chống HARKing. Đổi ngưỡng sau khi thấy kết quả phải ghi rõ trong paper là đã đổi.
- 6 · Kết quả chính thức của B-infer phải chạy với rescore (không `--no-rescore`)

Không rescore thì posterior thoái hoá thành hàm tất định của (s dự đoán, φ) và λ mất ý nghĩa.
- 7 · Mọi run ghi kèm `config` / `args` vào output JSON

Không ghi → không tái lập → không dùng được.

Bất biến vận hành bổ sung:

- **Idempotency:** `photos` và `--step all` bỏ qua việc đã xong; muốn chạy lại phải `--force` hoặc xoá output.
- **Thứ tự** `selftrain` → `infer` : pool_quads sinh bằng seed teacher chỉ là sơ bộ, phải đề lại bằng student cuối.
- **Provenance mềm:** không chuyển posterior thành bộ lọc cứng; quad `provenance_flip` vẫn giữ để audit.
- **Hai mẫu annotation tách biệt:** D0 (recall) ≠ AUDIT (precision) — trộn là circular.

19.5 Trạng thái & việc còn thiếu (theo mức độ chặn)

#	Việc	Chặn cái gì	Ưu tiên
1	Annotate D0 300 review (điền <code>gold_quads</code>) → dựng <code>outputs/drift/recall_table.json</code>	D-a (hiệu chỉnh recall) và ablation –D-a	P1
2	Hoàn thiện bước áp <code>recall_table</code> trong <code>module_d_drift</code> (hiện là stub chỉ log)	Kết quả cuối của Module D	P1
3	Chạy E1/E1b/E1c (<code>module_b_eval</code> + chrono probe)	Bảng F1 tham chiếu — mọi kết luận drift phải hiệu chỉnh mức tự tin theo E1c	P1
4	Chạy C1 train_verifier để biết GO/NO-GO	Scope của bài (Information Fusion vs IP&M/KBS)	P1
5	Quét λ ∈ {0,5...0,9} trên dev, tối đa dev F1 sentiment	λ=0,7 hiện là giá trị đặt tay, chưa chọn trên dev	P2
6	Annotate AUDIT 300 quad (<code>correct</code> 0/1)	Spearman go/no-go của bridge (cần ≥30 cặp khớp)	P2
7	Cài E3b (injection length/recall)	Bảng chứng cho D-a	P2
8	Cài 1[<code>span hợp lệ</code>] trong <code>apply_weights</code>	Công thức w_q đầy đủ như đặc tả	P2
9	Cài A7 langid thật (fasttext) → thêm trường <code>lang</code> vào store	Biến kiểm soát ngôn ngữ trong strata/phân tích	P3
10	Cài kênh semantic $E_{a,t}$ (opinion centroid, encoder đóng băng)	Phát hiện “vấn đề nhưng lý do đổi” (opinion drift)	P3
11	Chốt tần suất lỗi rơi phủ định trong gold (A9: 73 ca, 5 ca kiểm tay: 4 đúng / 1 sai)	Khai báo giới hạn dữ liệu	P3

Kaggle áp ba hạn chế lên pipeline: VRAM 16 GB (T4), phiên ~9–12 h, disk ~20 GB. Vast.ai bỏ cả ba. Mục này tách rõ **cái gì đổi vì KẾT QUẢ** đúng hơn với **cái gì đổi chỉ vì phần cứng** — và **cái gì Vast KHÔNG** giải quyết được.

Trước tiên: một lỗi cấu hình làm kiểm định KHÔNG THỂ dương tính

Permutation test không cho p nhỏ hơn $f = 1/(n_perm + 1)$. BH-FDR gán cho p ở hạng k giá trị $q_k = p_k \cdot m/k$, với m = số giả thuyết trong lưới. Suy ra: q nhỏ nhất đạt được ở hạng k là $f \cdot m/k$. Vậy để phát hiện được **MỘT aspect drift đơn lẻ** (k=1) — đúng thứ mà E3c tiêm vào và đúng thứ paper muốn kết luận — cần

$n_perm \geq m / \alpha - 1$

Với lưới chính: `--level taxonomy_code` cho tối đa **20 aspect** (14 `CODES_REPORTABLE` + 6 bucket category cho 17 code hiếm), lưới adj = π -adj U v-adj nên **m = 40** → $n_perm \geq 799$.

Kiểm bằng chính `U.benjamini-hochberg`, m = 40, $\alpha = 0,05$ — j = số aspect cùng drift:

n_perm	p sần	j=1 (đơn lẻ)	j=2	j=5	j=10
300 (mặc định CŨ của eval_injection)	0,00332	CHẾT q=0,133	CHẾT q=0,066	OK q=0,027	OK q=0,013
800	0,00125	OK q=0,050 (sát ngưỡng)	OK q=0,025	OK q=0,010	OK q=0,005
1000 (N_PERMUTATION hiện tại)	0,00100	OK q=0,040 (biên 1,25×)	OK q=0,020	OK q=0,008	OK q=0,004
5000 (đề xuất cho Vast)	0,00020	OK q=0,008	OK q=0,004	OK q=0,002	OK q=0,001

Hệ quả cụ thể: **E3c ở --n-perm 300 không thể thành công** — nó tiêm drift vào *đúng một aspect* (`AM_FOOD`), nên cột `valence_detected_within_3` bị ép về `false` ở **mọi** δ , không phải vì δ quá nhỏ mà vì số học. Nếu nó có dương tính thì là nhờ kênh π bị nhiễu kéo hạng BH lên — tức nhờ đúng cái confound của §14.2, không phải nhờ bằng chứng v.

Đã sửa trong code: mặc định `--n-perm` của `eval_injection` nâng 300 → `C.N_PERMUTATION` (1000); và Module D nay tự kiểm, log `ERROR` khi $f \cdot m > \alpha$ (với `--final` thì **dừng hẳn**), đồng thời ghi `fdr_grid_size`, `p_perm_floor`, `n_perm_min_for_grid`, `fdr_reachable_single_effect`, `min_simultaneous_effects_detectable` vào JSON output. **Đọc mấy trường đó trước khi tin `n_significant_after_fdr`.**

20.1 Đổi để KẾT QUẢ đúng hơn (thứ tự theo mức ảnh hưởng)

#	Tham số	Kaggle	Vast	Vì sao đây là vấn đề KHOA HỌC, không phải tốc độ
1	<code>--n-perm</code>	1000 (D) · 300 (inj)	5000	Xem hộp trên. 1000 chỉ hơn ngưỡng 799 khoảng 1,25× — nếu số aspect tăng lên 25 thì m=50 và n_perm=1000 rơi đúng biên. 5000 cho p sần 0,0002, dư biên và cho FDR đủ độ phân giải để xếp hạng các aspect, không chỉ để cắt ngưỡng.
2	<code>--n-boot</code>	0 (tắt)	1000	Mặc định đang TẮT bootstrap, nên <code>slope_ci95</code> không có trong output. Paper báo slope mà không có CI là điểm reviewer nhắm ngay. Đây là bước CPU, Vast rẻ.
3	cohort của <code>infer</code>	<code>--limit 200000</code>	bỏ --limit	Đo được: corpus có 1.790.037 đơn vị suy luận. Lấy 200k là 11% . Bỏ <code>--limit</code> thì hết cả phương sai lấy mẫu và hết mối lo mẫu lệch — kể cả sau khi đã đổi sang reservoir.
4	<code>MAX_TGT_TOKENS</code>	192	448	Khử một thiên lệch đã biết (§10.3): ở 192 token, unit từ ~9–10 quad bị truncate mất chính quad đang rescore → <code>p_model</code> uniform → posterior thành prior thuần. Gold có segment tới 23 quad × ~17 token ≈ 390. Phải nâng cho CẢ train VÀ infer rồi train lại — model train ở 192 sẽ không sinh quá 192 một cách đáng tin.
5	<code>--model</code>	<code>mt5-small</code>	<code>mt5-base</code> → thử <code>mt5-large</code>	<code>mt5-small</code> là mặc định của <code>orchestrator Kaggle</code> , không phải của <code>module_b_train</code> (vốn là <code>mt5-base</code>). Extractor tốt hơn → quad tốt hơn → mọi số hạ nguồn tốt hơn. Ghi rõ model nào trong paper.
6	λ (<code>--lam</code>)	0,7 đặt tay	quét trên dev	Đây là việc còn treo #5 trong §16 của chính spec này. $\lambda=0,7$ chưa từng được chọn trên dev. Quét $\lambda \in \{0,5 \dots 0,9\}$ và chọn theo dev F1 sentiment — bỏ được một tham số đặt tay, tức bỏ một chỗ reviewer bắt lỗi.
7	encoder Module C	<code>ViT-B-32</code>	<code>ViT-L-14</code>	Go/no-go của Module C gate trên $\Delta AUC \geq 0,05$. Với ViT-B-32, NO-GO có thể là do dung lượng encoder chứ không phải do “ảnh không mang tin hiệu” — hai kết luận rất khác nhau cho paper. Chạy cả hai và báo cáo ΔAUC của mỗi encoder mới là kiểm định công bằng.
8	<code>--rounds</code>	2	3–4	An toàn <i>sau khi</i> đã sửa tiêu chí dừng (§9.2): vòng mới chỉ được nhận khi vượt dev F1 tốt nhất, nên thêm vòng chỉ có thể tốt hơn hoặc dừng, không thể âm thầm xấu đi.
9	seed	1 seed	3–5 seed	Chạy lại toàn Module D với <code>--seed</code> khác nhau và báo cáo độ tản của <code>n_significant_after_fdr</code> . Rẻ (CPU) và trả lời trực tiếp câu “kết quả có phải do một lần bốc thăm may mắn không”.

20.2 Đổi vì phần cứng (KHÔNG ảnh hưởng con số)

Hạng mục	Hướng dẫn
----------	-----------

<code>--batch</code> / <code>--score-batch</code>	Bắt đầu từ mặc định của module (<code>32</code> / <code>48</code>), không từ override của Kaggle (<code>2</code> / <code>4</code>). Ràng buộc VRAM nằm ở <code>--score-batch</code> , không phải <code>--batch</code> : bước rescore dựng 3n chuỗi cho unit n quad, mỗi chuỗi dài n quad (§10.3). Cách hiệu chỉnh: chạy <code>--limit 2000</code> và theo dõi <code>nvidia-smi</code> , tăng dần tới ~80% VRAM. Đây là tham số duy nhất cần đo trên máy cụ thể — đừng chép số từ tài liệu.
Độ chính xác số	Giữ fp32 cho bước rescore. fp16/bf16 làm đổi log-prob → đổi <code>p_model</code> → đổi posterior → đổi <code>w</code> . Nếu muốn dùng half precision thì chỉ cho <code>generate</code> , và phải ghi rõ trong paper là hai bước dùng độ chính xác khác nhau.
Disk	Đo được cho corpus: pool thô 1,5 GB · store 179 MB · 119.620 review có ảnh (~ 14 GB nếu 120 KB/ảnh) · pool_quads corpus ~3,6M quad × 718 B ≈ 2,6 GB (nên ~0,6–0,8 GB) · checkpoint mỗi bản vài GB. Cấp ≥ 80 GB cho corpus + ảnh + vài checkpoint.
Tải ảnh	Bước network-bound, không phải GPU : corpus 119.620 ảnh @ <code>--sleep 0.4</code> = 13,3 h (T-unbiased: 13.039 ảnh = 1,4 h). Chạy song song với các bước GPU ngay từ đầu, đừng để nó nằm trên đường găng. Có thể giảm <code>--sleep</code> nhưng phải lịch sự với CDN.
RAM	<code>fit_bridge</code> nạp file quad vào RAM (đã thu gọn còn 11 trường, bỏ <code>text</code>). Vẫn nên fit bridge trên T-unbiased , không phải corpus. Cấp ≥ 32 GB RAM; 64 GB nếu muốn fit bridge trên corpus.
Không cần nữa	Không còn hạn mức phiên nên <code>--step all</code> chạy một lần là được. Cơ chế sentinel/skip vẫn hữu ích khi một step lỗi giữa đường — và nay đã kiểm file không rỗng (§17.6).

20.3 Vast KHÔNG giải quyết được (vẫn chặn số cuối)

Hạng mục	Chặn bởi	Cần gì
Khoá blocklist (<code>hotel,date</code>) (§5.2)	Thiếu dữ liệu	<code>metadata/reviews_with_dates.jsonl</code> trong <code>hamos-mabsa</code> . Không có thì <code>flag_gold_leak</code> = 1.466 thay vì 18.475 → ~17.000 dòng lẻ ra bị chặn đang lọt vào training . GPU mạnh hơn chỉ làm leakage lan nhanh hơn.
Probe E1c (§8.4)	Thiếu dữ liệu	Cùng file trên (0/10.711 segment có ngày)
D-a hiệu chỉnh recall (§13.2)	Thiếu annotation	Tập audit D0 (~300 review) → <code>recall_table.json</code> → viết bước áp thật . <code>--final</code> hiện chặn đúng để không sinh số cuối chưa hiệu chỉnh.
Go/no-go Spearman của Module C (§12)	Thiếu annotation	300 quad audit người gán. Nhánh này chưa chạy lần nào trên dữ liệu thật.
Độ phân giải của E4 (§14.3)	Thiết kế	Với ~20 aspect thì lưới quá nhỏ để ước lượng FPR. n_perm lớn không sửa được; cần nhiều cohort/level hoặc bootstrap ở mức aspect.

20.4 Lệnh chạy đề xuất trên Vast (corpus, số cuối)

```
##### 0. môi trường #####
export PRISM_TABSA_ROOT=/workspace/Prism/prism
export PRISM_HAMOS_ROOT=/workspace/hamos-mabsa
export PRISM_WORK_DIR=/workspace/outputs
export PRISM_MODEL_DIR=/workspace/models
export PYTHONPATH=$PRISM_TABSA_ROOT/src
export PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True
pip install -r requirements.txt          # có chặn trên version
pip freeze > requirements.lock.txt       # FREEZE rồi commit - artifact của paper
python -m unittest discover tests        # 70 test, ~0,15 s
python -m prism.smoke_test               # PHẢI PASS trước khi tốn GPU

##### 1. tải ảnh - chạy SONG SONG ngay từ đầu (13 h, không dùng GPU) #####
nohup python scripts/download_pool_photos.py --cohort corpus --limit 0 > photos.log 2&1 &

##### 2. store + data (CPU) #####
python -m prism.module_a_store
# KIỂM: store_report.json -> blacklist_date_key_active phải là true
python -m prism.module_b_data

##### 3. train seed (GPU) - mt5-base, token limit đã nâng #####
# Nâng MAX_SRC/MAX_TGT_TOKENS trong config.py lên 160/448 TRƯỚC khi train
python -m prism.module_b_train --model google/mt5-base --epochs 10 --batch 8 --grad-accum 4
python -m prism.module_b_eval --ckpt models/seed_extractor

##### 4. quét λ trên dev (bỏ tham số đặt tay) #####
for LAM in 0.5 0.6 0.7 0.8 0.9; do
    python -m prism.module_b_infer --ckpt models/seed_extractor --cohort B-anchor --limit 20000 --lam $LAM --sample-seed 1
    # chấm sentiment F1 trên dev, giữ λ tốt nhất
done

##### 5. self-train (GPU) #####
python -m prism.module_b_selftrain --rounds 4 --cohort B-anchor --batch 32 --score-batch 48 --infer-limit 400000

##### 6. infer TOÀN CORPUS - không --limit (GPU, bước dài nhất) #####
python -m prism.module_b_infer --ckpt models/selftrain_round<N> --cohort corpus --batch 32 --score-batch 48 --lam <λ đã quét>
# KIỂM log: tỷ lệ p_model_truncated phải ~0 sau khi nâng MAX_TGT_TOKENS

##### 7. Module C - chạy CẢ HAI encoder để ΔAUC là kiểm định công bằng #####
python -m prism.module_c_reliability --stage train_verifier # ViT-B-32
# rồi đổi model_name -> ViT-L-14, chạy lại, so ΔAUC của hai bản
python -m prism.module_c_reliability --stage apply_verifier --cohort T-unbiased
python -m prism.module_c_reliability --stage bridge --cohort T-unbiased --audit outputs/reliability/audit_sample_300.jsonl
python -m prism.module_c_reliability --stage apply --cohort corpus

##### 8. Module D - số cuối. --final chặn nếu D-a còn stub #####
python -m prism.module_d_drift --quads outputs/reliability/quads_weighted.jsonl.gz --level taxonomy_code --cohort corpus --n-perm 5000 --n-boot 1000 --final
# KIỂM output: fdr_reachable_single_effect=true · recall_adjusted=true

##### 9. cohort không thiên vị + độ tàn theo seed #####
python -m prism.module_d_drift --quads outputs/reliability/quads_weighted.jsonl.gz --level taxonomy_code --cohort T-unbiased --n-perm 5000 --n-boot 1000
for S in 1 2 3 4 5; do
    python -m prism.module_d_drift --quads outputs/reliability/quads_weighted.jsonl.gz --cohort corpus --n-perm 5000 --seed $S --out outputs/drift/seed_$S.jsonl
done

##### 10. injection & negative control - n-perm ĐỦ cho lưới #####
Q=outputs/reliability/quads_weighted.jsonl.gz
python -m prism.eval_injection --quads $Q --test composition --n-perm 5000
python -m prism.eval_injection --quads $Q --test valence --n-perm 5000
python -m prism.eval_injection --quads $Q --test shuffle --n-perm 5000 --repeats 10
```

Thứ tự ưu tiên nếu ngân sách GPU có hạn

Rẻ mà ảnh hưởng lớn nhất (chỉ CPU): #1 `--n-perm 5000` · #2 `--n-boot 1000` · #9 nhiều seed. Ba cái này không tốn GPU nào và sửa đúng chỗ để bị chất vấn nhất về mặt thống kê.

Tốn GPU, đáng nhất: #4 nâng `MAX_TGT_TOKENS` + train lại (khử thiên lệch đã biết) → #3 infer toàn corpus → #5 `mt5-base`.

Chỉ khi đã có annotation: #7 encoder Module C — không có 300 quad audit thì go/no-go vẫn không chấm được dù encoder tốt đến đâu.

Phụ lục A · Chỉ mục CLI đầy đủ

```
##### Kiểm tra trước tiên #####
$env:PYTHONPATH='src'
python -m unittest discover tests
python -m prism.smoke_test

##### Module A #####
python -m prism.module_a_store # không có cờ CLI

##### Module B #####
python -m prism.module_b_data # không có cờ CLI

python -m prism.module_b_train \
  [--model google/mt5-base] [--epochs 10] [--batch 8] [--grad-accum 4] [--lr 3e-4] \
  [--max-src 160] [--max-tgt 192] [--seed 20260821] \
  [--train-file outputs/extract/train.t2t.jsonl] [--dev-file outputs/extract/dev.t2t.jsonl] \
  [--out models/seed_extractor]

python -m prism.module_b_eval \
  [--ckpt models/seed_extractor] [--test-file outputs/extract/test.t2t.jsonl] \
  [--gold-file data/raw/test.jsonl] [--batch 16] [--max-tgt 192] \
  [--out-prefix outputs/extract/eval]

python -m prism.module_b_selftrain \
  [--rounds 2] [--cohort B-anchor] [--tau 0.7] [--tau-post 0.8] \
  [--max-ratio 3.0] [--max-skew 0.10] [--infer-limit 200000] \n  [--empty-ratio 0.1] [--batch 32] [--score-batch 48]

python -m prism.module_b_infer \
  [--ckpt models/seed_extractor] [--cohort T-unbiased|A-dense|B-anchor|corpus] \
  [--batch 32] [--score-batch 48] [--lam 0.7] [--no-rescore] \n  [--limit 0] [--sample-seed 20260821]

##### Ảnh + mẫu annotation #####
python scripts/download_pool_photos.py \
  [--cohort T-unbiased] [--limit 0] [--sleep 0.4] [--timeout 15]

python -m prism.make_audit_samples [--quads <pool_quads/*.jsonl.gz>] [--n 300] [--seed 20260821]

##### Module C (4 stage, đúng thứ tự) #####
python -m prism.module_c_reliability --stage train_verifier
python -m prism.module_c_reliability --stage apply_verifier \
  --quads outputs/extract/pool_quads.<cohort>.jsonl.gz \
  --image-index outputs/reliability/pool_image_index.json \
  --out outputs/reliability/pool_quads_vimg.<cohort>.jsonl.gz
python -m prism.module_c_reliability --stage bridge \
  --quads outputs/reliability/pool_quads_vimg.<cohort>.jsonl.gz \
  --audit outputs/reliability/audit_sample_300.jsonl
python -m prism.module_c_reliability --stage apply \
  --quads outputs/extract/pool_quads.<cohort>.jsonl.gz \
  --out outputs/reliability/quads_weighted.jsonl.gz

##### Module D (kết quả cuối) #####
python -m prism.module_d_drift --quads outputs/reliability/quads_weighted.jsonl.gz \
  [--level taxonomy_code|aspect_category] [--cohort corpus] \
  [--min-stratum 30] [--min-valence-w 10] [--n-perm 1000] [--n-boot 1000] \
  [--fdr 0.05] [--seed 20260821] [--out <path>] [--final]

##### Robustness #####
python -m prism.eval_injection --quads <file> --test composition [--n-perm 300] [--out-dir <dir>]
python -m prism.eval_injection --quads <file> --test valence [--aspect AM_FOOD] [--t0 2023-09]
python -m prism.eval_injection --quads <file> --test shuffle [--repeats 5]

##### Orchestrator Kaggle #####
python scripts/kaggle_pipeline.py --step <step|all> \
  [--repo-url ..] [--repo-dir /kaggle/working/Prism] \
  [--work-dir /kaggle/working/outputs] [--model-dir /kaggle/working/models] \
  [--cohort T-unbiased] [--ckpt <dir>] [--limit 0] [--model google/mt5-small] \
  [--epochs 10] [--batch 2] [--score-batch 4] [--grad-accum 8] [--rounds 2] \
  [--infer-batch 2] [--infer-score-batch 4] [--level taxonomy_code] \
  [--test composition] [--force]
```

Phụ lục B · Bản đồ file toàn repo

Đường dẫn	Nhiệm vụ
README.md	Cửa vào: cấu trúc repo, quick start, chuỗi Kaggle, danh mục tài liệu phương pháp

pyproject.toml	Packaging src-layout (prism-tabsa 0.1.0, requires-python ≥3.9, dependencies rỗng) + cấu hình pytest (testpaths=tests , pythonpath=src)
requirements.txt	Deps nặng phân nhóm theo module, có ghi chú nhóm nào cần cho gì
.gitignore	Loại /outputs/ , /models/* , /data/raw/*.jsonl — dữ liệu và checkpoint không commit
Dockerfile · Dockerfile.torch · docker-compose.yml	\$18.2
data/README.md	Mô tả 4 file raw + quy tắc blacklist; cảnh báo hotel_absa_labeled.jsonl không chứa quad
src/prism/	
config.py	\$2 — mọi hằng số/ngưỡng
utils.py	\$4 — I/O, parse VN, strata, CLR, BH-FDR, Welch-t, quad_uid
module_a_store.py	\$5
module_b_data.py	\$6 — linearize / parse_linearized (dùng lại ở infer & eval)
module_b_train.py	\$7
module_b_eval.py	\$8
module_b_selftrain.py	\$9
module_b_infer.py	\$10
module_c_reliability.py	\$12 — 4 stage + GO_NOGO
module_d_drift.py	\$13
eval_injection.py	\$14
make_audit_samples.py	\$11.2
smoke_test.py	\$19.2
README.md (trong package)	Runbook chi tiết từng bước + changelog audit code 2026-08-22 + 9 mục quy tắc
scripts/	
kaggle_pipeline.py	\$17 — orchestrator 1 file, 13 step
download_pool_photos.py	\$11.1
probe_complaint_composition.py	\$14.4 — CACT keyword probe (13 code, lexicon từ gold + seed thủ công)
docs/	
method_table_q1.md	Bảng method hợp nhất: module × contribution × novelty × rủi ro; evaluation/ablation/baseline; roadmap; venue
method_proposal_q1.md	Lập luận novelty + audit vòng 2 + chọn venue
method_spec_aspect_drift.md	Audit dữ liệu đầy đủ (§2 + Phụ lục A), formulation toán, evaluation protocol chi tiết
approach_temporal_trend.md	CACT — bằng chứng khả thi cho Module D (§15)
pipeline_io.md · repo_flow.md	Bảng I/O từng step · flow & nhiệm vụ từng file
prism_spec.html · prism_spec.pdf	Tài liệu này (HTML là nguồn, PDF là bản in)
kaggle_notebooks/ — 01→11 + 00_full_pipeline + README (§17.5) · tests/ — 6 file, 61 test (§19.1)	

Phụ lục C · Tra cứu số liệu đã đo

```
POOL  data/raw/hotel_booking_unlabeled.jsonl
1.949.604 review · 10.631 hotel · 2022-02→2025-03 (38 tháng) · 184 null date · 0 dòng JSON lỗi
duplicate 15.137 (0,78%)
text: both 678.194 (34,8%) · pos-only 444.508 (22,8%) · neg-only 27.713 (1,4%) · none 799.189 (41,0%)
    → universe text-bearing = 1.150.415 (59,0%)
photo 121.584 (6,24%) · stars_rating có ở 89,9%
review/hotel: mean 183,4 · median 66 · p90 457 · max 8.599
tháng phủ/hotel: mean 19,7 · median 19 · ≥24 tháng: 4.025 hotel · ≥36 tháng: 962 hotel
state:   Cặp đôi 40,8% · Gia đình 24,0% · Khách lẻ 18,9% · Nhóm 16,0%
country: VN 29,9% · Pháp 6,8% · Úc 6,6% · Đức 6,5% · Anh 6,4%
score:   điểm 10 = 44,9% · trung bình theo tháng chỉ dao động 8,47-8,87

GOLD  hamos-mabsa/data/annotations/quads.jsonl
23.995 quad · 12.601 segment · 8.796 review · 3.399 hotel · 9.219 ảnh (986 MB) · 38 tháng
category(6): FACILITY 55,1 · AMENITY 20,5 · SERVICE 10,7 · EXPERIENCE 8,3 · LOYALTY 3,2 · BRANDING 2,2
taxonomy(31): FAC_ROOM 5,444 (22,7%) ... BRA_REPUTE 22 (0,09%)   [9 code <100 quad · 17 code <300]
sentiment: pos 79,4% · neg 15,4% · neu 5,2% (1.248)
implicit: aspect 3,3% (782) · opinion 1,2% (283)
quad/review mean 2,73 (max 36) · quad/segment mean 1,90 (max 23) · segment mean 13 từ (max 147)
ngôn ngữ: en 7.473 · vi 1.032 · other 205 · đa ngữ 86           → 80,5% en · 16,2% vi
split review 6.156/1.320/1.320 · split segment 8.816/1.895/1.890 · hotel 2.381/504/514 (overlap 0)
multimodal: 100% quad có primary_image_id · 95,6% review 1 ảnh · 42,6% trải >1 code · 13,3% lẫn cục tính

OVERLAP  gold hotel ⊂ pool: 3.399/3.399 (100%) · pool review thuộc hotel gold: 1.249.424 (64,1%)
cohort A (gold ∧ ≥1000): 276 · B (gold ∧ ≥300): 1.221 · C (gold ∧ ≥100): 2.313

CONFOUNDER (đo theo tháng)
reviewer VN: 89,7% (2022-03) → 23,0% (2023-03) → 19,7% (2024-03) → 16,1% (2025-01)
% có text:   42,1% (2022-02) → 63,4% (2025-03)
độ dài TB:   27,5 từ → 40,8 từ
hotel/tháng: 549 (2022-02) → 8.535 (2025-02)
first-month: 549 hotel bắt đầu 2022-02 · 2.025 hotel bắt đầu 2022-03   (dấu hiệu truncation)

FIELD PROVENANCE (gold n=14.129 quad; mẫu score 400k review)
POS → (0,931 / 0,036 / 0,033)   NEG → (0,217 / 0,142 / 0,641)
pos-only n= 88.380 score TB 9,52 median 10 chỉ 1,2% có score ≤6
neg-only n= 5.839 score TB 5,33 median 5   59,0% có score ≤6
both      n=137.196 score TB 8,28
→ polarity field là nhãn sentiment gần-xác-định ⇒ (a) s gần như miễn phí;
   (b) review_score KHÔNG độc lập với nó ⇒ không được dùng cả hai cùng phía

COVARIATE SHIFT ẢNH (pool có text)
có ảnh n= 114.988 · 56,7 từ · score 8,90
không ảnh n=1.035.427 · 36,5 từ · score 8,68

RUN THẬT (2026-08-21 → 2026-09-07)
store:   đọc 1.949.604 → ghi 1.921.661 · has_text 1.141.532 · gold-leak flagged 18.475
        cohort 270 / 1.207 / 514 / 10.629
data:    16.789 + 3.600 + 3.606 = 23.995 quad · chrono 7.046 / 3.665 · roundtrip mismatch 0
train:   mt5-small · T4 · 10 epoch × ~397 s · best dev loss 0,190413 (epoch 9)
smoke:   blacklist 148 leak/20k · 36 kỳ · E4 shuffle FPR 0,0 (5 lượt) · 37 unit test PASS
```

Phụ lục D · Ký hiệu & thuật ngữ

Ký hiệu / thuật ngữ	Nghĩa
quad	(aspect_term, taxonomy_code, opinion_term, sentiment) + category suy ra từ code
segment	Đoạn văn bản trong một review — đơn vị extraction của gold
đơn vị suy luận	Cặp (một ô text của review, φ) — mỗi review pool cho tối đa 2 đơn vị
φ (phi)	Field provenance : ô nguồn của đoạn text — POS (review_positive) hoặc NEG (review_negative)
π _{a,t}	Share-of-complaints của aspect <i>a</i> ở kỳ <i>t</i> (phân tích trên thang CLR)
v _{a,t}	Negativity rate = P(negative nhắc <i>a</i> , kỳ <i>t</i>) — estimand trung tâm
E _{a,t}	Opinion centroid = Σw·ψ(opinion)/Σw với ψ = encoder đóng băng — kênh semantic chưa cài
raw / adj	Bản thô / bản đã direct standardization theo strata
D-a / D-b	Hiệu chỉnh recall (lỗi đo của công cụ) / hiệu chỉnh thành phần (đối tổng thể)
stratum	Ô phân tầng = (country_bloc, traveller_type) → 20 ô; +length_bin → 60 ô
cohort	Nhóm hotel dùng để lọc phân tích: A-dense · B-anchor · T-unbiased · corpus

CLR	Centered log-ratio — biến đổi bắt buộc cho dữ liệu compositional
BH-FDR	Benjamini–Hochberg false discovery rate, $\alpha = 0,05$
conf_seq	exp(trung bình log-prob token) của chuỗi target đã sinh
p_posterior	Xác suất hậu nghiệm của sentiment thắng sau khi kết hợp P_{model} với prior φ
provenance_flip	Cờ: posterior đổi sentiment so với model → nhóm giàu mìa mai/phủ định, giữ để audit
v_image	Điểm verifier V(ảnh, category) $\in [0,1]$ cho quad thuộc review có ảnh
$\hat{r}(q)$	Đầu ra bridge: xác suất dự đoán $V=1$ từ đặc trưng chi-text
w_q	Trọng số tin cậy cuối của quad = $1[\text{tax hợp lệ}] \cdot (1[\text{span hợp lệ}]) \cdot \hat{r}(q)$
go/no-go	Điểm quyết định cuối Module C: trượt ngưỡng → bỏ C, lùi về A+B+D
HARKing	Đặt giả thuyết sau khi biết kết quả — chống bằng cách chốt mọi ngưỡng trước khi chạy
IPW	Inverse propensity weighting — hiệu chỉnh covariate shift theo <code>has_photo</code>

PRISM — Đặc tả pipeline (bản chi tiết). Nguồn: `src/prism/*.py · scripts/*.py · tests/*.py · kaggle_notebooks/ · docs/{pipeline_io, repo_flow, method_table_q1, method_spec_aspect_drift, approach_temporal_trend}.md · outputs/store/store_report.json · outputs/extract/data_report.json · models/seed_extractor/training_log.json · Dockerfile* · docker-compose.yml`.
Commit gốc `fc5c774` (2026-09-07). Mọi vị trí được trích theo **tên module/hàm**, không dùng số dòng. Số liệu **[đo]** lấy từ audit dữ liệu và report của các run thật; ô **LỆCH** chưa cài / phản ánh đúng trạng thái code tại commit này. Tái tạo bản PDF này: `chrome --headless=new --no-pdf-header-footer --print-to-pdf=docs/prism_spec.pdf docs/prism_spec.html`.